

A
Project Report on
**“ AI Legal Assistant: A RAG-Based
Intelligent Legal Consultation System ”**

Submitted in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted by

**Soham Sunil Malvadkar (22141229)
Sanket Nivas Chopade (22141234)
Abhijeet Vilas Ganjave (22141236)
Akash Ananda Khot (22141244)
Sanket Nandkumar Adake (22143248)**

Under the Guidance of

Dr. S. A. Thorat



Government College of Engineering, Karad

(An Autonomous Institute of Government of Maharashtra)

Academic Year 2025-2026

(Semester-VIII)

Government College of Engineering, Karad
(An Autonomous Institute of Government of Maharashtra)



Department of Information Technology

CERTIFICATE

This is to certify that, the project entitled “**AI Legal Assistant: A RAG-Based Intelligent Legal Consultation System**” has been successfully completed by

Soham Sunil Malvadkar (22141229)

Sanket Nivas Chopade (22141234)

Abhijeet Vilas Ganjave (22141236)

Akash Ananda Khot (22141244)

Sanket Nandkumar Adake (22143248)

of Final Year B. Tech. I.T class under my supervision and guidance, in partial fulfillment of the requirement for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Information Technology from the Government College of Engineering, Karad for the academic Year 2025-26 Semester VIII.

Date:

Place: Karad

Dr. S. A. Thorat

Project Guide

Dr. R. B. Kulkarni

Head

Information Technology Department

External Examiner Sign

DECLARATION

We, declare that this written submission represents ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date:

Place: Karad

ACKNOWLEDGEMENT

Apart from individual efforts, the success of any project depends largely on the encouragement and guidelines of many others. We take this opportunity to express our gratitude to the people who have been instrumental throughout the project work.

It is our privilege to express our gratitude towards our project guide, **Dr. S. A. Thorat**, for his valuable guidance, encouragement, inspiration, and whole-hearted cooperation throughout the project work. We thank him for being a motivation through all our highs and importantly, our lows. His insights into natural language processing and legal technology have been invaluable in shaping this project.

We deeply express our sincere thanks to our **Head of Department, Dr. R. B. Kulkarni** for encouraging and allowing us to present the project on the topic “**AI Legal Assistant: A RAG-Based Intelligent Legal Consultation System**” and providing us with the necessary facilities to enable us to fulfill our project requirements as best as possible. We take this opportunity to thank all faculty members and staff of Department of Information Technology, who have directly or indirectly helped our project.

We pay our respects to honorable **Principal, Dr. S. J. Wagh** for his encouragement. Our thanks and appreciations also go to our family and friends, who have been a source of encouragement and inspiration throughout the duration of the project.

Date:

Place: Karad

ABSTRACT

The language of statutes, dispersion of legal materials, and cost of attorney's fees are barriers to access to laws. This work introduces the AI Legal Assistant, a multi-module intelligent system that utilizes RAG to make legal knowledge accessible to everyone. The integrated components of the system include Legal Chatbot based on curated Indian statutes on Cyber Law, Marriage Law, Employment Law, and Consumer Law. Document Analyzer that retrieves, summarizes, and answers questions about legal contracts that users upload, and Lawyer Finder that searches for vetted lawyers by geography, practice area, and rates for consultations. Developed with a Streamlit web interface, it utilizes HuggingFace sentence-transformers for embedding generation, FAISS for efficient vector similarity search, and Groq LLaMA-3 for high-throughput inference. Test on 100 queries in four domains show that the chatbot achieves a precision of 97% with a response latency of 3.2 seconds on average. 80% clause-level accuracy was achieved from digital pdfs in document analysis. The system offers an open-source, low-resource template for making preliminary legal guidance broadly available.

Keywords: Retrieval-Augmented Generation, FAISS, LangChain, Groq LLM, HuggingFace, Indian Law, Streamlit, NLP, Sentence Transformers, Vector Search, Legal Chatbot, Document Analysis.

INDEX

Sr. No.	Contents	Page No.
I	List of Figures	i
II	List of Tables	ii
III	Abbreviations	iii
Chapter 1	Introduction & Motivation	1
1.1	Overview of Indian Legal Landscape	1
1.2	Motivation for the Project	2
1.3	Problem Statement and Objectives	4
1.4	Scope and Applications	5
Chapter 2	Literature Survey	7
2.1	Introduction	7
2.2	Chatbots and AI in Legal Research	7
2.3	NLP-Based Approaches to Legal Text Processing	10
2.4	Retrieval-Augmented Generation (RAG)	11
2.5	Vector Similarity Search and FAISS	13
2.6	Feature Comparison with Existing Systems	14
2.7	Identification of Research Gaps	16
Chapter 3	Software Requirements Specification	17
3.1	Purpose of the System	17
3.2	Overall Description	18
3.3	Functional Requirements	21
3.4	Non-Functional Requirements	25
Chapter 4	Design & Methodology	28
4.1	Proposed System Architecture	28
4.2	Data Flow and UML Diagrams	31
4.3	Module-Level Design	39
4.4	Project Plan - Estimation and Schedule	44
4.5	Design Decisions and Rationale	46
Chapter 5	Implementation	48

5.1	Overview of Project Modules	48
5.2	Module 1: Legal Chatbot Implementation	49
5.3	Module 2: Document Analyzer Implementation	52
5.4	Module 3: Lawyer Finder Implementation	55
5.5	Algorithm Details	56
5.6	Testing and Validation	61
5.7	Implementation of the Proposed System	62
Chapter 6	Results and Discussion	65
6.1	Experimental Setup	65
6.2	Chatbot Precision and Performance	67
6.3	Document Analyzer Performance	71
6.4	Lawyer Finder Relevance Evaluation	73
6.5	Discussion of Results	75
Chapter 7	Conclusion and Future Scope	78
7.1	Conclusion	78
7.2	Future Scope	80
	References	82
	Annexure	
1	Plagiarism Report of Black Book	
2	Research Paper Attachment	
3	Proof of Research Paper Submission	

List of Figures

3.1	System Context Diagram: AI Legal Assistant in its Operating Environment	18
4.1	Three-Tier System Architecture of the AI Legal Assistant	29
4.2	Data Flow Diagram (Level 0) - Context View	31
4.3	Use Case Diagram - Actor-System Interactions	33
4.4	Document Analyzer Module - Processing Workflow	35
4.5	Legal Chatbot Module - RAG Processing Workflow	37
4.6	Lawyer Finder Module - Filtering and Ranking Workflow	38
5.1	Sliding-Window Chunking Algorithm	57
5.2	FAISS Semantic Retrieval Algorithm	58
5.3	Lawyer Filtering and Composite Ranking Algorithm	61

List of Tables

2.1	Comparative Analysis of Legal AI Systems	15
3.1	Product Functions by Module	19
3.2	User Categories and Characteristics	20
3.3	Functional Requirements: Document Analyzer	22
3.4	Functional Requirements: Legal Chatbot	23
3.5	Functional Requirements: Lawyer Finder	24
3.6	Functional Requirements: User Interface	24
4.1	Knowledge Base: Statute Corpus Composition	40
4.2	Lawyer Database Schema	43
4.3	Project Development Schedule	44
4.4	Technology Stack Summary	45
6.1	Chatbot Precision and Latency by Legal Domain	68
6.2	Response Latency Breakdown by Processing Stage	69
6.3	Precision Comparison: RAG System vs. Retrieval-Free Baseline	70
6.4	Document Extraction Accuracy by Document Type	72
6.5	Lawyer Finder Relevance Evaluation	74
6.6	Sensitivity Analysis of Composite Scoring Weights	75

ABBREVIATIONS

AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
PDF	Portable Document Format
API	Application Programming Interface
UI	User Interface
FAISS	Facebook AI Similarity Search
KB	Knowledge Base
ML	Machine Learning
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
HTTP	HyperText Transfer Protocol
SRS	Software Requirement Specification
CPU	Central Processing Unit
ITA	Information Technology Act, 2000
CPA	Consumer Protection Act, 2019
HMA	Hindu Marriage Act, 1955
SMA	Special Marriage Act, 1954
IDA	Industrial Disputes Act, 1947
MBA	Maternity Benefit Act, 1961
DPDPA	Digital Personal Data Protection Act, 2023
DPI	Dots Per Inch
SBERT	Sentence-BERT
LPU	Language Processing Unit

Chapter 1

Introduction & Motivation

1.1 Overview of Indian Legal Landscape

Indian Legal system is one of the largest and most complicated systems in the world, with more than 1250 central Acts, countless state laws, and a continuously evolving matrix of judicial precedents. In the face of such a sprawling statutory terrain, the gulf between what we have on paper and what the common man can access in the way of legal knowledge remains unbridgeable. It is apparent that the language of legislative instruments is naturally ambiguously layered with arcane terms and complex sentence formations, which render understanding difficult even for educated people, not to mention the illiterate.

Accordingly, the Indian legal system is among the largest and most complex in the world, with over 1,500 central laws, countless state laws, and a matrix of judicial precedents that is constantly evolving. Such an all-encompassing legislative landscape notwithstanding, there prevails a deep embarrassment of disconnects between what exists legally and what is actually possible in the way of legal empowerment for the common man. The language of the legislations is naturally a very terse one and contains archaic words and complex sentence structures, which makes it difficult even for well-educated people required to interpret the laws, not to mention the common man.

A 2023 poll, conducted by the Daksh India Foundation, suggested that less than 15% of

litigant in rural areas were able to correctly identify the legal provision applicable to their particular dispute without the aid of a professional. This figure illustrates a deep-rooted failure in the dissemination of legal literacy. Most Indians, in turn, either rely on informal community networks — where legal myths and misinformation go uncorrected — or give up on remedies altogether when they encounter the system’s bewildering complexity.

There are three main barriers that contribute to this gap in access. First, the complexity of the law is a daunting barrier. The specialized terminology and complex sentence construction of legal materials form a barrier that is virtually impenetrable to non-attorneys. Secondly, the sheer volume and scattering of laws and regulations at central, state, and even district levels make it virtually impossible for anyone who lacks specialized training to make sense of it all. Third, by no means less important, the financial element must be factored in the average fee for a single consultation with an advocate in one of India’s cities is anything between INR 500 and INR 5,000, making professional legal advice unaffordable for a large part of the populace.

The rise of digital technologies and the advent of artificial intelligence potentially provide a promising avenue for closing this gap. Yet the use of these technologies in the law is equally in need of attention to its own concerns, with respect to truth and trust. Generic large language models (LLMs), such as GPT-3 or ChatGPT, although powerful in NLP applications, are at high risk of hallucination, which means generating factually non-correct but plausible legal citations or interpretations. In the legal domain, such bugs might have disastrous real-world consequences and cause users to make ill-advised decisions about penalties set by statute, filing deadlines, or criteria for eligibility.

1.2 Motivation for the Project

What drives us to develop the AI Legal Assistant is the coming together of technological possibilities and urgent public needs. The conception and development of such a system was motivated by a number of considerations.

The opening up of legal knowledge is central to the motivation. In a rule-of-law

democracy, access to legal information should not be a tightly held secret that only the wealthy owners of professional advice can access. This project seeks to enable citizens to take control of their rights and duties as citizens of the country of law by developing an AI-based platform that converts legalese and complex statutory language into plain explanations.

The misinformation/hallucination problem of modern AI chatbots is another big factor. Traditional chatbots, as well as general LLMs, often produce replies that sound authoritative and contain factual errors, fake case citations, or wrong citations to statutes. For anyone seeking true legal advice, it's nearly impossible to know what part of what they're being told is wrong information or lies told with confidence. The Retrieval-Augmented Generation framework provides a technique to address this issue by basing each answer on an evidence document.

The inclusion of a separate Document Analyzer module was also motivated by the fact that reading a document is time-consuming. Agreements have traditionally been legal agreements, contracts of employment, and notices to consumers are many pages long, crammed with legal jargon. For someone with no legal background, identifying core duties, rights, and liabilities in these documents can take hours, if not days. A system that could automatically summarise the key provisions and respond to users' specific queries regarding content would be a game-changer.

A further gap that this project aims to alleviate is the lack of cohesive platforms integrating self-service legal information with options for professional advice. Most of the tools provide one or the other - automated information retrieval or lawyer directories but they rarely do both in the same interface. The addition of a Lawyer Finder module is a recognition that, although AI can help make people more legally literate, there are still times when they need to speak to a professional lawyer.

1.3 Problem Statement and Objectives

1.3.1 Problem Statement

To design and develop an AI-based Legal Assistant System for analysing legal documents and to answer the queries related to Indian Marriage law, Employment Law, Cyber Law and Consumer law accurately and with a context-based grounding, and to allow users to find appropriate lawyers through a searchable and filterable directory. The system will use Retrieval-Augmented Generation, vector embeddings, and large language model inference to limit hallucination and improve factual reliability.

1.3.2 Primary Objectives

The project pursues the following primary objectives:

1. Legal Chatbot Development: Towards developing a country- specific conversational agent for the domain of Indian law- Marriage Law (Hindu Marriage Act, 1955 and Special Marriage Act, 1954), Employment Law (Industrial Disputes Act 1947, Maternity Benefit Act 1961, Code on Wages 2019), Cyber Law (Information Technology Act 2000 along with the 2008 amendment and related rules), and Consumer Law (Consumer Protection Act 2019 and E-Commerce Rules 2020) - which fetches pertinent statutory paragraphs and produces precise, citation corroborated responses.

2. Document Analysis Automation: To implement an intelligent document processing pipeline that extracts textual content from both born-digital and scanned PDF documents, generates hierarchical embeddings, indexes content using FAISS for efficient similarity search, and responds to natural language queries regarding document content, including summarisation, clause-specific questions, and obligation extraction.

3. Hallucination Mitigation: To employ a Retrieval-Augmented Generation architecture that grounds every generated response in retrieved statutory text or document content, supplemented by post-generation validation checks for citation accuracy and out-of-domain speculation detection.

4. Lawyer Discovery Platform: A verified legal professional to be built to be able to be searched, by practice area, locality, level of experience and fee for consultation, structured and filterable, which will provide a way to connect automated legal information with professional legal advice.

5. Accessible Web Interface: All these should be implemented for system delivery via a web application based on Streamlit (which requires no client-side installation), accessible using standard web browsers, with an intuitive user interface that can be used by users in nontechnical fields.

1.3.3 Secondary Objectives

Besides the main aim, the project is also endeavouring to achieve the following: multi-format document ingestion (such as image-based PDFs that will require OCR processing), secure processing of user documents but without any persistent storage, the use of FAISS indexing to optimise search latency, the design of a modular architecture that would allow for future extensions to different legal areas, and the laying down of the groundwork for one day supporting queries in multiple languages.

1.4 Scope and Applications

1.4.1 In-Scope Functionality

The T-scale for the AI Legal Assistant includes Parsing and understanding legal texts in PDFs and plain text, creating document summaries and answers to document-specific questions; a domain-trained chatbot (limited to four distinct areas of Indian law), a searchable lawyer directory with filters for specialization, experience, rating, location, and consultation fees; and delivery via a web-based Streamlit interface.

1.4.2 Out-of-Scope Items

The system explicitly disclaims offering any professional legal advice, which could substitute for a conversation with a licensed attorney. It's currently limited by the four

mentioned areas and doesn't include all Indian laws as of now. It doesn't generate legally binding documents or provide live chat with attorneys. The existing version works only in English.

1.4.3 Applications

The service is geared to several user communities, including ordinary citizens who want a basic overview of the law as it relates to their personal or work lives students and scholars who need to locate pieces of statutory content members of the military and others who need to rapidly understand legal text as reading material (for example, before they sign contracts) entrepreneurs and HR representatives who want a high-level understanding of employment regulation and people hoping to find and interact with lawyers that could help them with their case.

Chapter 2

Literature Survey

2.1 Introduction

The AI Legal Assistant is based on a rich body of research from a variety of fields: AI and law, natural language processing of statutory text, retrieval-augmented generation methods, and semantic search using vectors. This chapter reviews the literature, discusses the contributions and gaps of prior work, and motivates the current system.

The literature review is divided along contextual, thematic lines and evolves from early chatbot models used in a legal context to more recent retrieval-augmented techniques which underpin this project's methodology. Relevance to the challenges of Indian legal information access and to the design choices that the AI Legal Assistant embodies are the criteria by which each considered work is evaluated.

2.2 Chatbots and AI in Legal Research

2.2.1 Early Legal Chatbot Systems

The use of conversational agents to provide legal information has been investigated for more than a decade, with initial systems targeting mainly rule-based question-answering in tightly scoped domains. These state-of-the-art systems demonstrated the feasibility of automated legal assistance, although their use was limited to brittle rule engines that

were unable to capture the variations in natural language legal queries.

Arora et al. (2025) presented an in-depth analysis of the role of natural language processing and AI chatbots in legal recruiting, indicating robust market enthusiasm for automated contract analysis solutions. Their prototype system, designed for legal hiring applications, used language models to determine candidate qualifications and assign candidates to suitable job positions. Yet, the system was based solely on the model’s parametric knowledge, learned through pretraining, and did not have a retrieval module to ground (or justify) the outputs in authoritative legal documents. This architectural design led to two major issues: 1) factual drift, in which the generated responses progressively drifted away from truthful legal contents in long conversations, and 2) jurisdictional errors, where the system produced occasional references to foreign (and thus irrelevant) legal provisions, a by-product of the worldwide distribution of its training data. These results emphasize the need for retrieval-based grounding, which was a major design principle in the current project.

2.2.2 Domain-Specific Legal Advisory Systems

An NLP-based chatbot for answering general legal questions in the Indian scenario. Kumar and Singh (2024) proposed a chatbot named LEGALBOT based on NLP for general legal enquiries in the Indian scenario. Employing natural language understanding pipelines, the system parses the user question and responds with relevant information fed into a curated knowledge base on Indian legal matters. LEGALBOT obtained a high level of response coherence, with users reporting a high level of comprehension of the provided legal explanations. However, the architecture of the framework was not designed to support vector index, it was based on keyword-matching and rule-based retrieval methods. This limitation resulted in two critical failures: First, the accuracy of retrieval at the statute-level dropped off significantly when queries were expressed in synonymous words or paraphrases of legal terminology instead of using the exact language of the statute. Second, it was not enabled to analyse particular documents provided by users,

which meant that it could not offer advice that was contextualised by such documents as agreements or contracts.

A systematic review of the capabilities, architectures, and application areas Sharma et al. (2024) surveyed AI-driven conversational agents in the legal field, enumerating their functionalities, structures, and areas of application. One major and enduring gap identified in their analysis is that the legal services for none of the identified solutions include a seamless transition from front-end AI-powered legal triage to human legal assistance. The authors stressed that the lack of such a bridge between the automated and professional services was a significant barrier because users who, through interacting with AI, found that their particular circumstances required professional assistance were not provided a means to transition. This finding directly led to the addition of the Lawyer Finder module to the AI Legal Assistant, designed as an intentional device for closing the loop between a self-service information one-stop shopping experience and the prospect of actually obtaining professional services through the lawyer finder.

2.2.3 Commercial and International Legal AI Platforms

A few pay-to-use platforms have been built in this space their utility for the Indian environment is limited. LawDroid and DoNotPay are two well-known examples that provide a means of filling out forms and procedural advice for routine legal work like fighting parking tickets or creating standard contract forms. These platforms, while illustrating that legal AI is commercially viable, have a few common attributes which make them not particularly relevant as prototypes for Indian legal aid - Their services are primarily paywalled: a model difficult to replicate for serving a poor clientele; Their knowledge frameworks are built around Western jurisdictions, primarily United States and European Union law, with no space for Indian statutory constructs; and They are not architected to support queries on unstructured documents - they rely on a structured form filling workflow.

2.3 NLP-Based Approaches to Legal Text Processing

2.3.1 Transformer Models for Legal Language

Transformer-based architectures had a disruptive effect on legal text processing. BERT and its derivatives have been shown to be the best in capturing semantic features of legal language, and they achieved the best performance over previous RNN-based models on various tasks, including statutory clause classification and judgment outcome prediction. The native bidirectional attention mechanism in these architectures is especially useful for legal text, in which meaning of a word is often entailed by its left and right contexts over multiple sentences.

Nevertheless, generic pre-trained models have been shown to be limited when directly used for legal text. Legal language was significantly different from the general-domain text that most models had been trained on, in that it contained specialised vocabulary, phrases, and collocations, as well as archaic constructions, and also used domain-specific syntactic structures. This move across domains encouraged the investigation of the legal-specific pre-training and fine-tuning methods, as well as the retrieval-augmented paradigm that bases model-produced results on domain-specific reference materials, rather than using only parametric knowledge.

2.3.2 Sentence Embeddings for Semantic Search

Reimers and Gurevych (2019) presented Sentence-BERT, a novel BERT-based embedding method that can be used to derive semantically meaningful fixed-sized sentence embeddings that can be compared using cosine-similarity and can also be used for clustering and information retrieval. Prior to this work, extracting sentence-level representations from BERT necessitated computationally intensive cross-encoder techniques that were unfeasible for large-scale retrieval. Sentence-BERT is based on siamese and triplet networks structures, and during fine-tuning, it can generate embeddings in which semantically similar sentences are close to one another in the vector space, while dissimilar sentences are

far apart.

The real-world consequences of this for our current project are direct and significant. The all-MiniLM-L6-v2 model used in the AI Legal Assistant is a compressed version of the Sentence-BERT architecture, fine-tuned through knowledge distillation to offer a good trade-off between embedding quality and speed. This yields very compact yet semantically rich 384-dimensional embeddings, which both fit efficiently in memory and retain an enough level of representational richness to capture the semantic content of legal passages. The choice of this particular embedding model was motivated by experimental results that indicated the small improvements in accuracy with larger embedding models were not worth the additional computational cost and latency in the target deployment environment.

2.4 Retrieval-Augmented Generation (RAG)

2.4.1 Foundations of RAG

Lewis et al. (2020) proposed the Retrieval-Augmented Generation architecture in their seminal work, demonstrating that involving a dense passage retriever with a sequence-to-sequence generator can achieve new state-of-the-art results on knowledge-intensive natural language processing tasks over purely parametric models. From a high level, this architecture can be described as a two-stage process: a query encoder that first transforms the input question into a dense vector, and then uses this as a query to achieve nearest-neighbor search to retrieve relevant passages from an external knowledge corpus, and a sequence-to-sequence generator. Second, a generator model is given the retrieved passages concatenated with the original query, and outputs a final answer conditioned on both the query and retrieved passages.

The intuition behind RAG is that language models, no matter how large, have a limited amount of parametric knowledge that is fixed at training time, and cannot be updated, corrected, or expanded without re-computationally expensive retraining. By external-

ising knowledge in a retrievable corpus, RAG-based systems can access knowledge that postdates training, they can integrate domain-specific knowledge without fine-tuning, and finally, they can cite source documents that users can analyze. These features are exactly what the legal information system needs, where factual correctness, jurisdiction-specificity, and the ability to trace back to an authoritative source are non-negotiable preambles.

2.4.2 RAG in High-Stakes Domains

Follow-up studies have subsequently generalized the RAG model to high-stakes application domains where factual trustworthiness has life-altering implications. In healthcare, RAG systems have been used to base clinical decision support on peer-reviewed literature and treatment guidelines. In finance, retrieval-augmented methodologies facilitate compliance verification against dynamic regulatory regimes. In all of these cases, the core value prop of RAG is the same: mitigating hallucination by restricting generation to information that can be linked to specific, retrievable source passages.

RAG for Indian legal advisory is a natural progression for this line of inquiry. The complexity of the Indian legal system-its multiple layers of jurisdiction, its frequent amendments, its complex cross-referencing between statutes-makes it a prime candidate for retrieval-augmented methods. The AI Legal Assistant uses RAG, it keeps a handpicked corpus of statutory documents, it encodes each passage into the FAISS vector store and fetches the most semantically relevant passages for each user query, and then generates the final response using the Groq LLM.

2.4.3 Evidence of RAG Effectiveness

The practical case for RAG for legal applications is also bolstered by quantitative results from the evaluation led by this project. RAG vs. baseline: With the same set of 100 legal queries, a vanilla GPT-3.5-turbo model without retrieval augmentation (RAG) was tested for baseline comparison. The baseline scored a precision of only 74%, while the

RAG-augmented system achieved a precision of 97%. This 23-percentage-point increase illustrates how useful retrieval grounding is in practice. The baseline model’s errors can be classified into two buckets: hallucinating non-existent statutory provisions and borrowing general legal principles from outside jurisdictions and their applications that are inconsistent with the specifics of Indian statutory provisions. The retrieval module indeed prevents both failure modes by limiting the generative scope of the model to information that is explicitly contained in the retrieved passages.

2.5 Vector Similarity Search and FAISS

2.5.1 The Challenge of Efficient Nearest-Neighbour Search

The Problem of Fast Nearest-Neighbour Search for High-dimensional Data The RAG architecture’s retrieval stage needs to find, out of millions or more of text passages, those that are most semantically related to the input query. The naïve approach of calculating cosine similarity between the query embedding and all the passage embeddings becomes computationally prohibitive when the corpus is large, causing a bottleneck that would make real-time retrieval impossible for applications that require user interaction.

Johnson et al. (2019) to overcome this problem with IndexIVFFlat, a simple and efficient index. FAISS provides several index types suited for different accuracy-speed trade-offs, as well as key solutions for high-dimensional, large-scale retrieval such as the inverted file with product quantization. The inverted file divides the vector space into Voronoi cells using k-means clustering and confines search during retrieval to one or a few cells rather than the entire institution. Product quantization compresses vector representations even further, resulting in vastly reduced memory requirements, as well as highly efficient distance computation via lookup tables.

2.5.2 FAISS for Real-Time RAG

In the AI Legal Assistant, FAISS has two purposes. For the Statutory module, a static pre-built FAISS index with all statutory chunks embeddings is distributed, allowing retrieval with sub-15-millisecond latency on CPU-only hardware. This speed is essential to sustain the interactive response times that users expect from conversational interfaces—the FAISS retrieval accounts for less than 0.5% of the overall end-to-end query processing time, so that the retrieval step is never the bottleneck.

Session-specific FAISS indices for the Document Analyzer are then built on the fly when you upload your document, which allows for the indexing of the embedded chunks from every user’s document. This per-session indexing design fulfills performance and privacy goals: it makes sure that retrieval runs on a corpus size that is both manageable and specific to the user’s document, and it keeps users’ data completely isolated with no persistent storage of uploaded content.

2.6 Feature Comparison with Existing Systems

Table 2.1 solidifies the comparison between existing legal AI systems and the proposed platform and showcases the distinctive capabilities of the AI Legal Assistant.

Table 2.1: Comparative Analysis of Legal AI Systems

System	Key Features	Limitations	RAG	Doc Analysis	Lawyer Finder
NyayGuru	Basic legal Q&A	No document analysis, no retrieval grounding	No	No	No
FreeLawyer AI	LLM-based legal interface	No PDF support, no retrieval augmentation	No	No	No
LawDroid / DoNotPay	Form filling, document templates	Not India-specific, paid access, Western jurisdictions only	No	Limited	No
Generic LLMs (GPT-4, etc.)	General question answering	Hallucination risk, no legal specialization, jurisdictional errors	No	No	No
LEGALBOT (2024)	NLP-based advisory for Indian law	No retrieval grounding, no document analysis capability	No	No	No
Arise Legal Bot	Conversational legal queries	Limited statutory coverage, no document processing	No	No	No
AI Legal Assistant (Proposed)	Chatbot + Document Analyzer + Lawyer Finder, 4 law domains	97% precision, 3.2s latency, RAG-grounded responses	Yes	Yes	Yes

2.7 Identification of Research Gaps

The related work findings bring to light multiple challenges, which the AI Legal Assistant is precisely tailored for. Firstly, the existing Indian legal chatbots tend to be ungrounded in retrieval, making them prone to hallucination and jurisdictional errors, which can endanger the lives of misguiding users. None of the systems we have seen combines document analysis with general legal query answering: users generally use one tool to ‘read’ their documents, and another to learn about general legal matters. Third, currently, there are no features to help users discover a lawyer from within these platforms, so those who decide they need professional assistance are forced to leave the platform and search for legal representation. Finally, the use of paid APIs and proprietary infrastructure for many systems impedes the possibility of deployment in resource-limited environments, such as those found in the Indian public service sector.

Through its tri-module architecture, the AI Legal Assistant bridges these gaps: the RAG-based chatbot ensures grounded, citation-backed responses, the Document Analyzer offers customized document comprehension with the same interface, the Lawyer Finder facilitates the transition from information to professional contact, and its open-source, Streamlit-based deployment allows for easy, browser-based access with no client-side installation required.

Chapter 3

Software Requirements Specification

3.1 Purpose of the System

The AI Legal Assistant is intended as an end-to-end solution accessible via the web that aims to make the extensive statutory matrix in India more accessible to the information needs of the layperson. The platform has been designed with three clear objectives in mind, matching its modular design.

Firstly, the legal document analysis is automated—including the extraction of text from PDFs of legal documents uploaded by the user, text splitting into semantically meaningful chunks, generation of dense vector embeddings, and natural language search over the documents. Automation transforms a traditionally all-day-long manual-reading chore into a task completed in seconds.

Second, the platform provides a user-friendly conversational interface that allows users to obtain precise domain-specific legal knowledge based on Retrieval-Augmented Generation. By limiting responses to retrieved statutory passages from curated collections on Marriage Law, Employment Law, Cyber Law, and Consumer Law, the tool alleviates the hallucination issue that tends to affect generic large language models when queried on legal matters.

Third, users can find professional legal advice through a comprehensive, searchable, and filterable list of vetted attorneys. This module recognizes that while AI can make legal

literacy significantly better, there are instances where human consultation is necessary, and the transition between automated and professional support should be seamless.

3.2 Overall Description

3.2.1 Product Perspective

The AI Legal Assistant is developed as a single-page application in a client-server architecture following a three-tier pattern separating presentation, business logic, and data persistence. Figure 3.1 illustrates a broad contextual perspective of the system in relation to its environment.

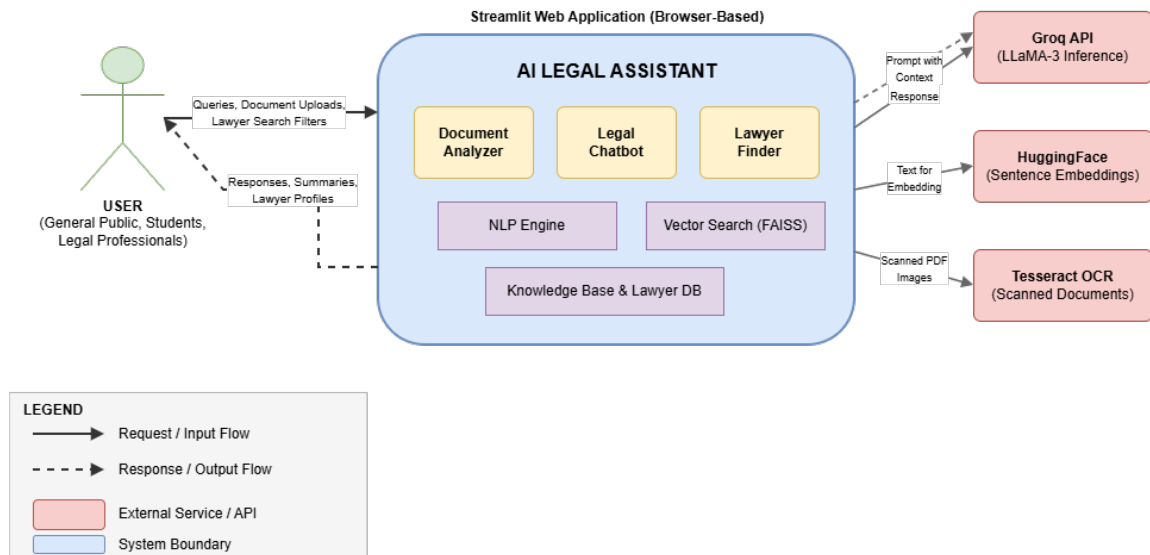


Figure 3.1: System Context Diagram: AI Legal Assistant in its Operating Environment

The frontend tier is built using Streamlit, a Python framework enabling declarative web app creation without specialized frontend development skills while maintaining responsiveness and browser accessibility. The application layer contains the three core business modules—Legal Chatbot, Document Analyzer, and Lawyer Finder—along with shared services for embedding generation, vector search, and LLM inference. The data tier consists of the FAISS vector store for statutory and document embeddings, the pandas-powered lawyer directory, and the curated corpus of statutory PDFs.

Integration with third-party services occurs through controlled interfaces: Hugging-

Face for embedding generation, Groq API for high-throughput LLM inference, and Tesseract OCR for scanned document processing. These dependencies are encapsulated behind abstraction layers to facilitate replacement if service availability or requirements change.

3.2.2 Product Functions

Table 3.1 enumerates the principal functions organized by system module.

Table 3.1: Product Functions by Module

Module	Key Functions
Document Analyzer	Accept document uploads (PDF, TXT) • Extract text via PyMuPDF or Tesseract OCR • Segment text into overlapping chunks • Generate FAISS index and respond to queries • Produce structured summaries identifying parties, obligations, and key terms
Legal Chatbot	Accept natural language queries across four legal domains • Retrieve top-k relevant statutory passages via FAISS • Construct grounded prompts with context and history • Generate responses via Groq LLaMA-3 with citations • Apply post-generation validation for hallucination and citation verification
Lawyer Finder	Display lawyer profiles with professional information • Filter by practice area, location, experience, and fee range • Rank results using weighted composite scoring • Render matched profiles as interactive card components
User Interface	Sidebar-based navigation between modules • Maintain session state for conversation continuity • Display safety warnings for unverified responses • Present analysis results with source passage highlighting

3.2.3 User Characteristics

The system accommodates diverse users with varying technical and legal expertise. Table 3.2 categorizes the anticipated user types.

Table 3.2: User Categories and Characteristics

User Category	Characteristics and Needs
General Public	Non-technical users with limited legal vocabulary; require simplified explanations; may upload rental agreements, employment contracts, or consumer notices for review
Students & Researchers	Use platform for academic reference and statutory lookup; benefit from citation-backed responses; may query across multiple domains
Small Business Owners	Need rapid comprehension of commercial agreements and employment law obligations; benefit from document summarisation and obligation extraction
Legal Professionals (Beginners)	Use as preliminary research aid to locate relevant statutory provisions; value retrieval speed compared to manual legal research
Rural and Under-served Users	May have limited internet connectivity or older devices; require lightweight interface with fast response times; benefit from free, browser-based access

3.2.4 Constraints

The system operates under several design and operational constraints:

Domain Coverage Constraint: The Legal Chatbot’s knowledge is currently limited to four legal domains—Marriage Law, Employment Law, Cyber Law, and Consumer Law. Queries on criminal law, property law, taxation, or constitutional law fall outside the system’s scope and should be explicitly declined rather than risk producing incorrect responses.

Connectivity Dependency: The system requires an active internet connection for LLM inference via the Groq API. While the Streamlit application could theoretically be served locally, the generative components depend on cloud-based inference services. Offline functionality is not supported in the current implementation.

Language Limitation: The platform operates exclusively in English for both input processing and response generation, representing a significant accessibility limitation given India’s linguistic diversity where a substantial portion of the population prefers regional languages.

Processing Time Constraint: Large documents exceeding 50 pages may require extended processing times for text extraction, chunking, and embedding generation. The system should communicate processing status to users during these operations.

Legal Advice Disclaimer: The system provides legal information, not legal advice. All responses must include appropriate disclaimers, and the platform must not be positioned as a substitute for professional legal consultation.

3.2.5 Assumptions and Dependencies

The system's design and operation are based on the following assumptions:

- Users will upload documents they have legal rights to access and have obtained legitimately.
- The Groq API will maintain availability during expected usage periods, with response times consistent with published service level objectives.
- The curated statutory corpus accurately reflects legislation as published in official government gazettes.
- Users possess basic digital literacy sufficient to navigate a web interface and formulate queries in English.
- The deployment environment provides Python 3.10+ with sufficient memory to load embedding models and maintain session-specific FAISS indices.

3.3 Functional Requirements

3.3.1 Document Analyzer Module

Table [3.3](#) specifies the functional requirements for the Document Analyzer module.

Table 3.3: Functional Requirements: Document Analyzer

FR ID	Requirement Description	Priority
FR-DA-01	Accept document uploads in PDF (.pdf) and plain text (.txt) formats via Streamlit file uploader.	High
FR-DA-02	Extract textual content from born-digital PDFs using PyMuPDF (fitz) library, preserving paragraph structure where possible.	High
FR-DA-03	Detect scanned/image-based PDFs (threshold: <100 chars/page) and automatically invoke Tesseract OCR at 300 DPI.	High
FR-DA-04	Preprocess extracted text by removing diacritics, headers, footers, and de-hyphenating split words.	Medium
FR-DA-05	Segment preprocessed text into chunks of ~800 tokens with 100-token overlap using sliding-window algorithm.	High
FR-DA-06	Generate 384-dimensional dense vector embeddings using all-MiniLM-L6-v2 from HuggingFace.	High
FR-DA-07	Construct session-specific FAISS IndexFlatL2 index from generated embeddings with unique session identifier.	High
FR-DA-08	Support three query archetypes: document summarization, clause-specific questions, and obligation extraction.	High
FR-DA-09	For summarization: retrieve top-10 chunks and construct structured prompt (parties, obligations, payment, dispute resolution, termination).	Medium
FR-DA-10	Generate responses via Groq LLM conditioned on retrieved document chunks with source references.	High
FR-DA-11	Do not persistently store uploaded documents or extracted text beyond active session lifetime.	High

3.3.2 Legal Chatbot Module

Table 3.4 specifies the functional requirements for the Legal Chatbot module.

Table 3.4: Functional Requirements: Legal Chatbot

FR ID	Requirement Description	Priority
FR-LC-01	Accept natural language queries from users through a text input interface.	High
FR-LC-02	Encode user queries into 384-dimensional dense vectors using all-MiniLM-L6-v2 model.	High
FR-LC-03	Retrieve top-5 most semantically similar statutory chunks from pre-built FAISS index for selected domain.	High
FR-LC-04	Maintain rolling conversation buffer of last 5 exchanges in session state for anaphoric reference resolution.	Medium
FR-LC-05	Construct multi-constraint prompt: (a) answer from context only, (b) cite sections, (c) include disclaimer, (d) acknowledge insufficient context.	High
FR-LC-06	Generate responses via Groq LLaMA-3 with temperature 0.1 to minimise stochastic variation.	High
FR-LC-07	Detect hallucination via speculation markers ("I believe", "typically", "usually") and display yellow warning.	Medium
FR-LC-08	Verify citations against lookup table of valid act-section pairs; display red alert for invalid citations.	Medium
FR-LC-09	Clear conversation history on domain switch or manual reset.	High
FR-LC-10	Support four legal domains: Marriage Law, Employment Law, Cyber Law, Consumer Law with dropdown selection.	High

3.3.3 Lawyer Finder Module

Table 3.5 specifies the functional requirements for the Lawyer Finder module.

Table 3.5: Functional Requirements: Lawyer Finder

FR ID	Requirement Description	Priority
FR-LF-01	Load curated lawyer database from CSV file containing 200 verified profiles with 9 attributes each.	High
FR-LF-02	Display lawyer profiles as styled cards: name, practice area, experience, rating, fee range, location, summary.	High
FR-LF-03	Provide sidebar filtering widgets: practice area (dropdown, 12-area taxonomy), location (text/dropdown), experience (slider), fee (range slider), rating (slider).	High
FR-LF-04	Apply all active filters simultaneously as compound boolean mask on DataFrame using vectorised operations.	High
FR-LF-05	Rank results: $\text{Score} = 0.40 \times \text{NormRating} + 0.35 \times \text{NormExp} + 0.25 \times (1 - \text{NormFee})$, min-max normalised.	Medium
FR-LF-06	Display top-5 ranked profiles by default with option to view additional results.	Medium
FR-LF-07	Include "Contact" action button for each profile (integration deferred to future).	Low
FR-LF-08	Display appropriate message when no lawyers match specified filter criteria.	High

3.3.4 User Interface Requirements

Table 3.6 specifies the functional requirements for the user interface.

Table 3.6: Functional Requirements: User Interface

FR ID	Requirement Description	Priority
FR-UI-01	Provide sidebar-based navigation enabling users to switch between three modules without page reload.	High
FR-UI-02	Implement lazy loading of modules using Python's importlib to minimise startup memory footprint.	Medium
FR-UI-03	Maintain consistent visual styling (colours, fonts, spacing) across all modules.	Low
FR-UI-04	Display processing indicators (spinners, progress bars) during operations exceeding 1 second.	Medium
FR-UI-05	Render Document Analyzer and Legal Chatbot in distinct tabs/sections within their respective module pages.	Medium
FR-UI-06	Display legal disclaimers prominently on home page and within chatbot response areas.	High

3.4 Non-Functional Requirements

3.4.1 Performance Requirements

- **Response Latency:** The Legal Chatbot shall produce complete responses within 5 seconds for 95% of queries under normal operating conditions, measured from query submission to final token delivery.
- **Document Processing Time:** The Document Analyzer shall complete text extraction, chunking, and index construction within 20 seconds for documents up to 50 pages.
- **FAISS Retrieval Speed:** Vector similarity search shall return top-k results within 200 milliseconds for indices containing up to 1,000 chunks.
- **Embedding Generation:** Sentence embedding for a single query shall complete within 200 milliseconds.
- **Concurrent Session Support:** The system shall support at least 10 concurrent user sessions without per-session response time degradation exceeding 20%.

3.4.2 Security Requirements

- **Document Confidentiality:** User-uploaded documents shall not be written to persistent storage. Document text and embeddings shall exist only in memory during the active session and shall be irretrievably discarded upon session termination.
- **API Key Protection:** Groq API credentials and other service keys shall be stored in a separate configuration file excluded from version control, never hard-coded in source files.
- **Data Isolation:** Session-specific FAISS indices shall be associated with unique session identifiers to prevent cross-user data leakage.

- **No External Data Transmission:** User documents and queries shall not be transmitted to any third-party service beyond Groq API (inference) and HuggingFace API (embedding). Document content sent externally shall be limited to retrieved chunks relevant to the specific query.
- **Input Validation:** All file uploads shall be validated for allowed MIME types and size limits before processing to prevent malicious file injection.

3.4.3 Usability Requirements

- **Learnability:** A first-time user with basic web browsing experience shall be able to submit a legal query or upload a document for analysis without consulting external documentation.
- **Error Messaging:** All error conditions shall produce descriptive messages in plain language explaining the error nature and suggesting corrective action where applicable.
- **Responsive Design:** The interface shall remain functional and readable across viewport widths from 768 pixels (tablet) to 1920 pixels (desktop).
- **Consistency:** Navigation elements, button placements, and interaction patterns shall remain consistent across all three modules.

3.4.4 Reliability Requirements

- **Availability:** The Streamlit application shall maintain 95% uptime during intended usage hours when deployed on the target hosting platform.
- **Graceful Degradation:** If the Groq API becomes unavailable, the system shall display a clear status message to users rather than failing with unhandled exceptions.
- **Error Recovery:** In the event of an LLM inference failure, the system shall automatically retry once before returning an error message to the user.

- **Invalid Input Handling:** The system shall gracefully handle corrupted PDFs, unsupported file formats, and empty queries by displaying appropriate error messages without crashing.

3.4.5 Maintainability Requirements

- **Modular Architecture:** Each of the three core modules shall be implemented in separate Python files with well-defined interfaces, enabling independent modification and testing.
- **Configuration Externalisation:** All configurable parameters (chunk sizes, retrieval k-values, model names, API endpoints) shall be defined in a centralized configuration file.
- **Code Documentation:** All public functions shall include docstrings describing parameters, return values, and raised exceptions.
- **Domain Extensibility:** Adding a new legal domain shall require only: (a) addition of new statutory PDFs to the corpus directory, (b) execution of the re-indexing script, and (c) registration of the new domain in the configuration

3.4.6 Scalability Requirements

- **Corpus Growth:** The FAISS indexing and retrieval pipeline shall maintain sub-200ms retrieval latency as the statutory corpus expands up to 5,000 chunks (approximately $8\times$ current size).
- **Lawyer Database Expansion:** The Lawyer Finder's pandas-based filtering shall scale linearly to at least 10,000 lawyer profiles without query-time degradation exceeding 100ms.
- **Concurrent Users:** The architecture shall support horizontal scaling through deployment of additional Streamlit server instances behind a load balancer without architectural modification.

Chapter 4

Design & Methodology

4.1 Proposed System Architecture

4.1.1 Architectural Overview

The AILA is designed with a modular structure as a three-tier web-based application that separates presentation, business logic, and data persistence layers. This design pattern has been chosen as it best suits the project's goals in terms of maintainability, extensibility, and deployment flexibility. The three-tier architecture allows each tier to evolve separately - the user interface may be improved without changing the core retrieval mechanism, new (legal) domains may be introduced in the data tier without changing the application layer, and new embedding or inference backends can be introduced without changing the presentation.

Figure [4.1](#) presents the comprehensive three-tier architecture, illustrating the constituent components within each tier and the communication pathways between them.

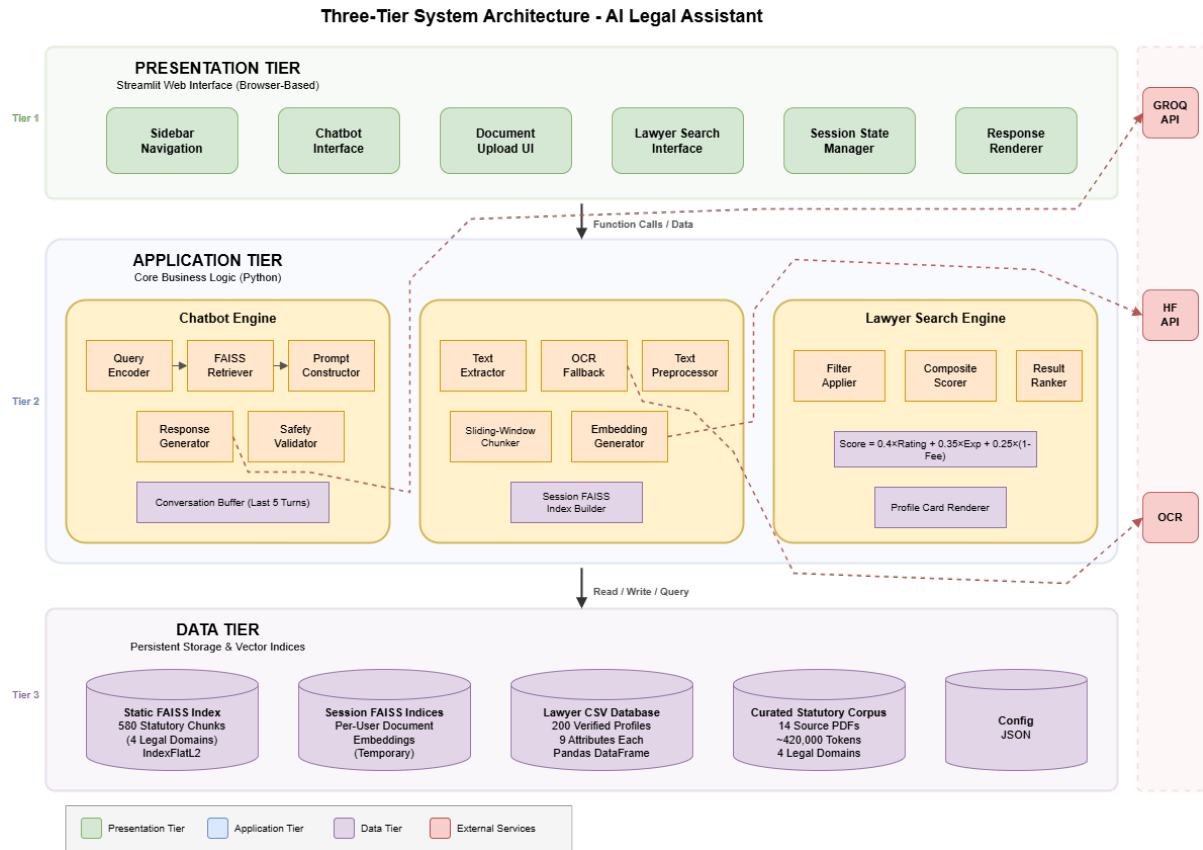


Figure 4.1: Three-Tier System Architecture of the AI Legal Assistant

The **Presentation Tier** is built on Streamlit, a Python-powered declarative framework that hides the complexity of HTML, CSS, and JavaScript yet enables building a responsive web application. Streamlit’s state facility is used to preserve conversation history, active FAISS indices, and module selection state across a user session, without having to persist session data on the server. Only through Python function calls does the presentation tier communicate to the application tier, both tiers run as separate threads of the same process - a design that avoids network overhead among tier communication, while retaining logical separations.

The **Application Tier** contains the three main modules - Chatbot Engine, Document Processing Pipeline, Lawyer Search Engine — as well as the cross-cutting services for embedding generation, prompt construction, and safety evaluation. Each module is encapsulated in its own Python file with clear function-based interfaces, allowing for modular development, testing, and future extensions. The application layer uses lazy

loading facilities provided by Python’s `importlib`, deferring module initialisation until the first time that the user accesses the corresponding feature of your application, hence improving startup time by reducing memory footprint.

The **Data Tier** is responsible for the following types of persistent and semi-persistent data. The static Faiss index is already populated with 384-dimensional embeddings of around 580 statutory chunks from 14 source PDFs within four legal domains, offline and loaded at application startup. Session-based faiss indices for the documents that users upload are built on-the-fly and held only in memory for the duration of the session, to provide data isolation between sessions. The lawyer database is kept as a CSV file, which is then imported into a pandas DataFrame, allowing for vectorised querying. The curated statutory corpus is the ground-truth source for all retrieval procedures, and original PDFs are stored for citation verification.

4.1.2 Module Interaction Patterns

The application-tier modules are organised according to the nature of their operations into three types of interacting modules. The Chatbot Engine is based on a synchronous request-response paradigm: a query from a user results in embedding generation, retrieval of nearest neighbors from FAISS, prompt construction, LLM inference, safety checking as a sequential pipeline with the final reply returned to the presentation tier for rendering. The Document Processing Pipeline establishes an initialization-query relation: upon uploading a document, a one-time processing routine (extraction, chunking, embedding, indexing) commences, and then, the session-local index is able to entertain multiple querying operations until the session is ended. The Lawyer Search Engine relies on a filter-rank-render approach: the user selects some criteria from the side bar widgets, and these are gathered into a compound boolean mask, then this is applied all at once in a vectorised manner to the DataFrame, and the ranked subset is rendered using card components.

Cross-module interactions are purposely minimised to retain modular autonomy. The

Chatbot and Document Analyzer have different indices, but use the same embedding model (all-MiniLM-L6-v2) and FAISS infrastructure. The Lawyer Finder is a standalone component with no ties to the NLP pipeline. This loose coupling guarantees that changes in one module will not have ripple effects in other modules.

4.2 Data Flow and UML Diagrams

4.2.1 Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram (Context Diagram) presented in Figure 4.2 models the system as a single process interacting with five external entities. This high-abstraction view establishes the system boundary and identifies the primary information flows crossing that boundary.

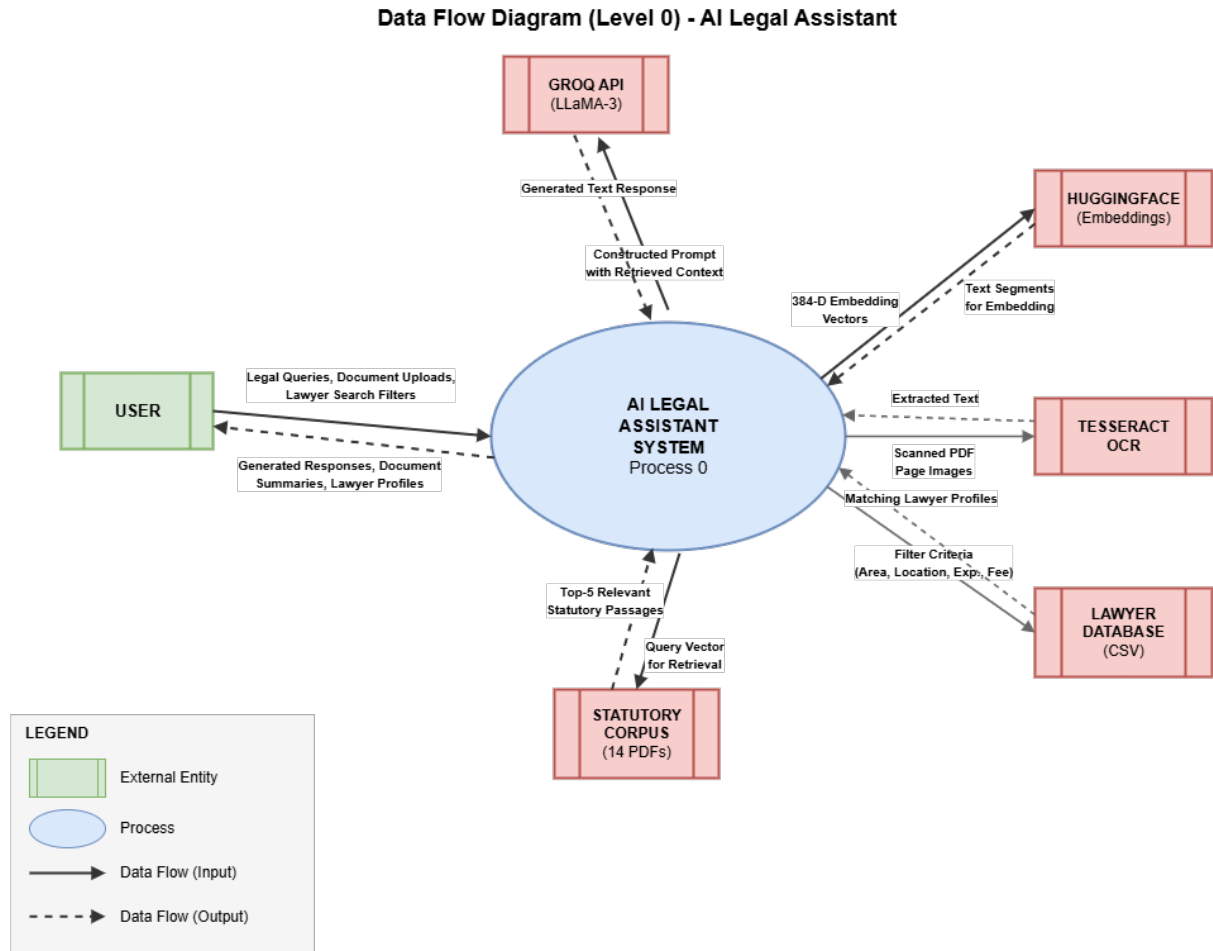


Figure 4.2: Data Flow Diagram (Level 0) - Context View

Five external entities interface with the system. The **User** submits legal queries in natural language, uploads documents for analysis, and specifies lawyer search criteria (practice area, location, experience range, fee range) in return, the system delivers generated responses with statutory citations, document summaries and clause-specific answers, and ranked lawyer profiles. The **Groq API** receives constructed prompts containing retrieved context and system instructions, returning generated text responses used for chatbot answers and document analysis outputs. The **HuggingFace Model Service** receives text segments for embedding, returning 384-dimensional dense vectors used for both statutory and document indexing. The **Tesseract OCR** service receives page images from scanned PDFs and returns extracted text when the primary PyMuPDF extraction yields insufficient content. The **Statutory Corpus** serves as the persistent knowledge base from which relevant passages are retrieved during chatbot query processing.

4.2.2 Use Case Diagram

The use case diagram in Figure 4.3 identifies three actor categories and their respective interactions with the system's functional modules.

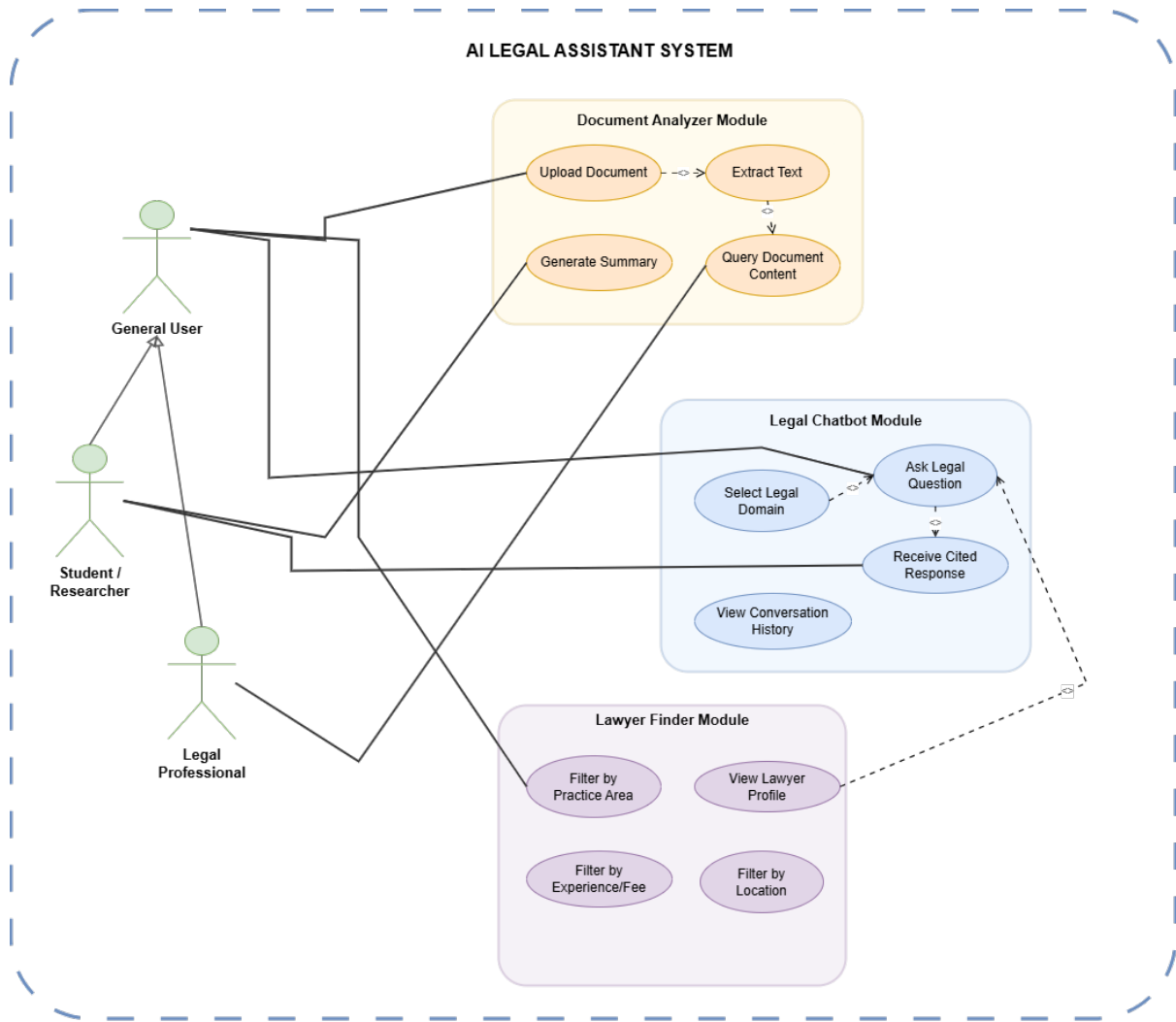


Figure 4.3: Use Case Diagram - Actor-System Interactions

The **General User** scenario encompasses all users who have access to the three sub-systems. This user may upload documents, submit legal questions in any of four areas of law, obtain summaries, and even search for lawyers based on an array of criteria. The **Student/Researcher** user is focused on the chatbot and document analysis scenarios, and in particular on providing responses that can be backed up by citations and on retrieving statute references for academic use. The **LegalProfessional** actor is a document analysis application focused on high-speed contract review and clause searching for professional legal users. Both Student/Researcher and Legal Professional are specialized forms of the General User they are extended versions of the base actor with its strength or weaknesses inherited and modified to fit their purpose.

4.2.3 Detailed Workflow Diagrams

Below are flowcharts showing in detail each of the three core modules, detailing the stepwise procedures of each pipeline.

Document Analyzer Workflow

Figure 4.4 shows the complete processing pipeline of the Document Analyzer module, starting from the upload of a single document to the generation of a response to a query.

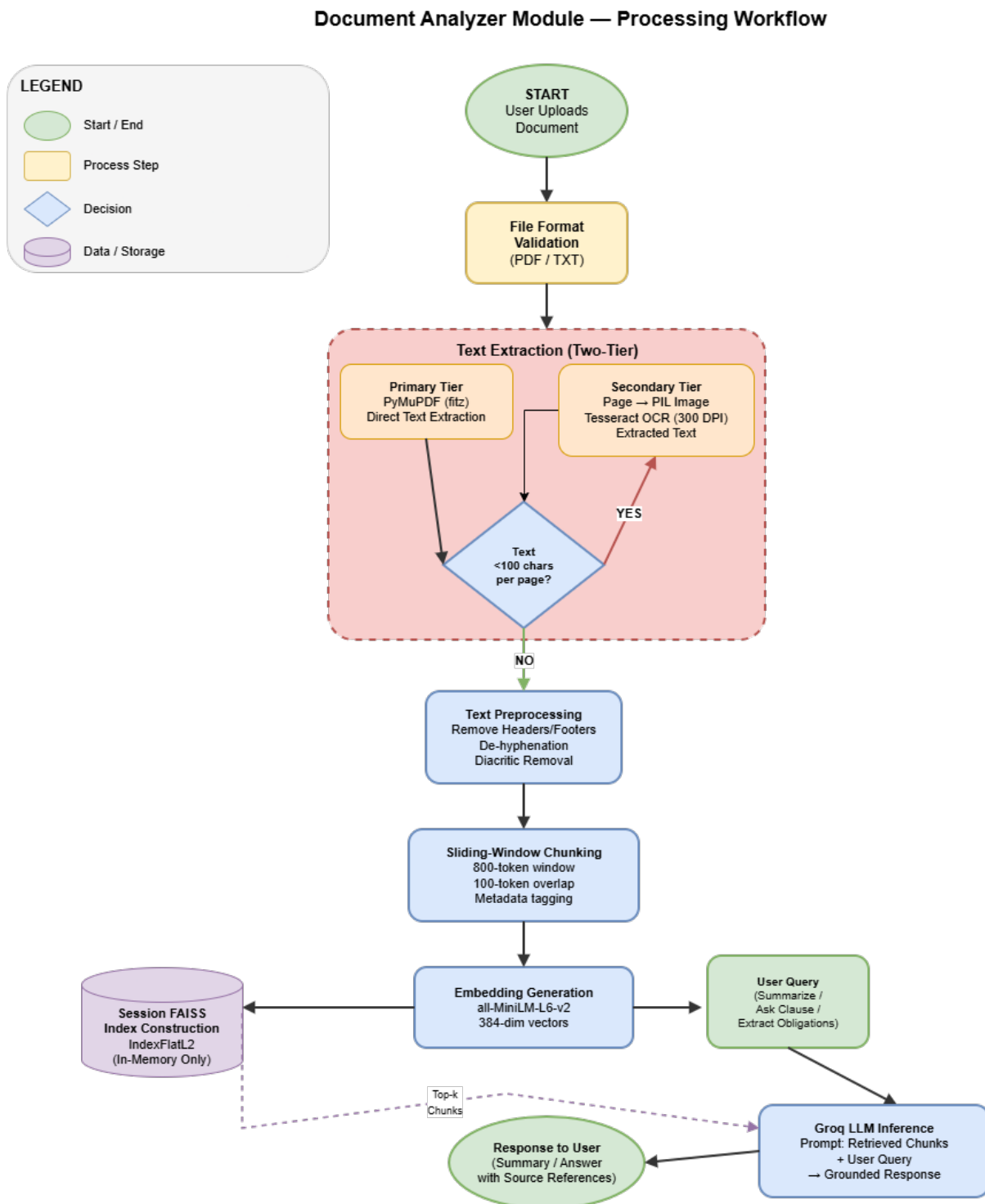


Figure 4.4: Document Analyzer Module - Processing Workflow

The workflow is triggered when a document is uploaded by the user with the Streamlit file uploader widget. The system checks the file format (PDF or plain text) and calls the corresponding extraction route. For born-digital PDFs, PyMuPDF (fitz) extracts text on a high level while on the fly preserving paragraph structure to the extent that

it is detectable. For any documents that per-page character output chunks size goes below a 100-character threshold - a telltale sign of a scanned or image-based PDF - the system renders each page to a 300 DPI PIL image and calls Tesseract OCR. The text extracted from any source is preprocessed for headers, footers, diacritics, and line-break hyphenation.

The preprocessed text is divided into segments by a sliding-window chunker with a window size of 800 tokens and an overlap of 100 tokens between each neighbouring chunk. This overlap guarantees that information that is semantically relevant and crosses the boundaries of two chunks can be recovered as a unit of context. Each chunk is labeled with source documents metadata, including document name and character-offset range for citation traceability.

Embedding generation utilizes the HuggingFace all-MiniLM-L6-v2 model, which returns 384 -dimensional dense vectors for each chunk. These vectors are stored in a session-specific FAISS IndexFlatL2 that is kept in RAM. Then, when the user later issues a query, the system encodes the query text using the same embedding model, finds the top-k most semantically similar chunks using FAISS nearest-neighbor search, then organizes a structured prompt with the retrieved context and the query archetype, and finally calls Groq LLM to generate the answer.

Legal Chatbot Workflow

Figure 4.5 presents the retrieval-augmented generation pipeline that constitutes the Legal Chatbot's core processing loop.

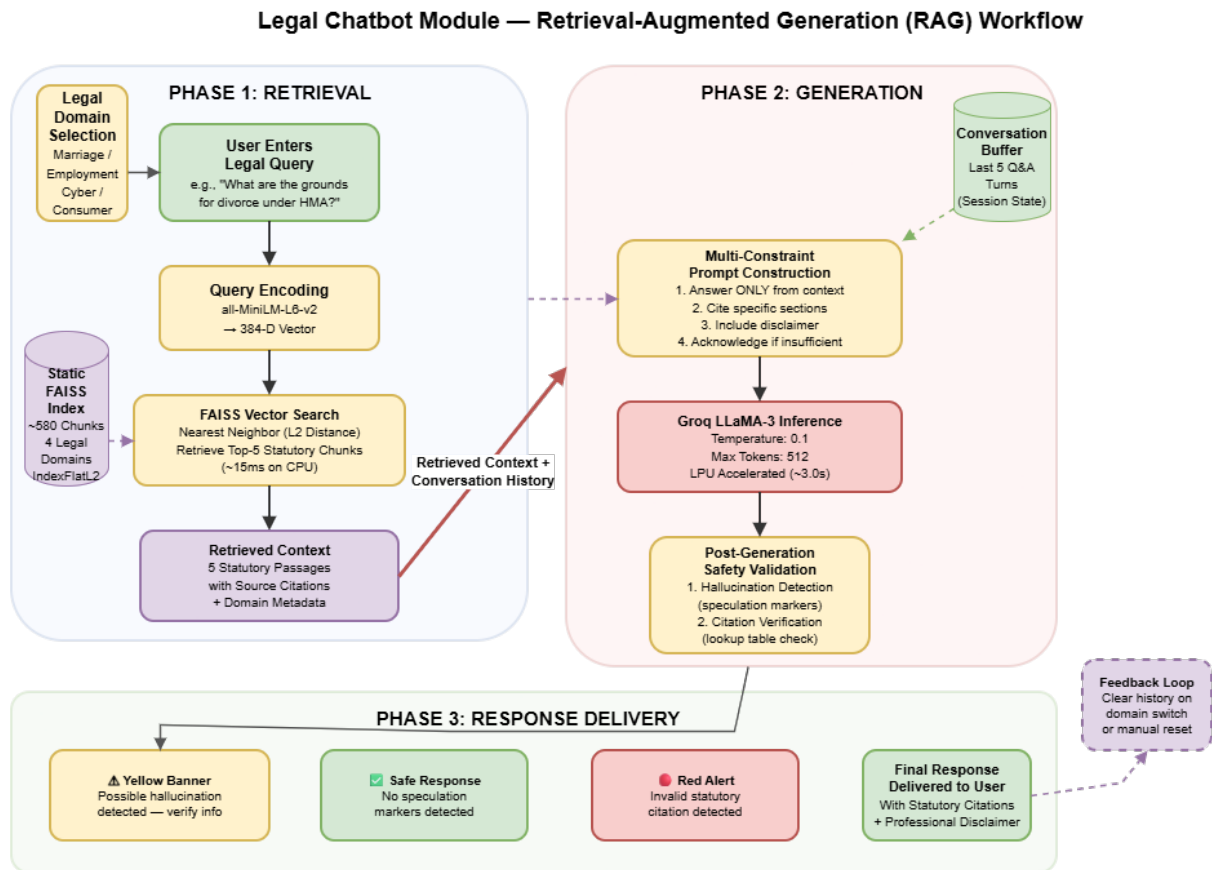


Figure 4.5: Legal Chatbot Module - RAG Processing Workflow

The user opens the chatbot workflow by choosing a legal area (Marriage Law, Employment Law, Cyber Law, or Consumer Law) and issuing a natural language query. The query text is transformed into a 384-dimensional dense vector by using the all-MiniLM-L6-v2 model. The query vector is then used to query the pre-built static FAISS index based on the domain to retrieve the top-5 nearest statutory chunks that are semantically closest in meaning.

The ChatGPT chunks retrieved, as well as the rolling conversation buffer of the last five exchanges (captured in the Streamlit session state), are then built into a multi-constraint prompt template. The template tells the Groq LLaMA-3 model to: (a) derive its answer solely from the given context passages, (b) cite specific statutory sections, if applicable, (c) include a disclaimer that suggests users seek professional legal advice, and (d) state when the retrieved context is not enough to fully answer the query. Model Response Temperature is set to 0.1, to reduce stochastic variation in response.

Before being delivered to the user, the response undergoes two validation tests post-generation. A heuristic hallucination detector looks for hedging markers of speculative language (e.g., “I believe”, “typically”, “usually”), which can be seen as hints that content not supported by the retrieved evidence is being generated by the model. A yellow warning banner is appended to the reply if confirmed. A citation validation procedure cross-checks any cited statute-subsection pairs with a table of predetermined valid pairs. Invalid citations trigger a red alert banner.

Lawyer Finder Workflow

Figure 4.6 illustrates the filtering and ranking pipeline for the Lawyer Finder module.

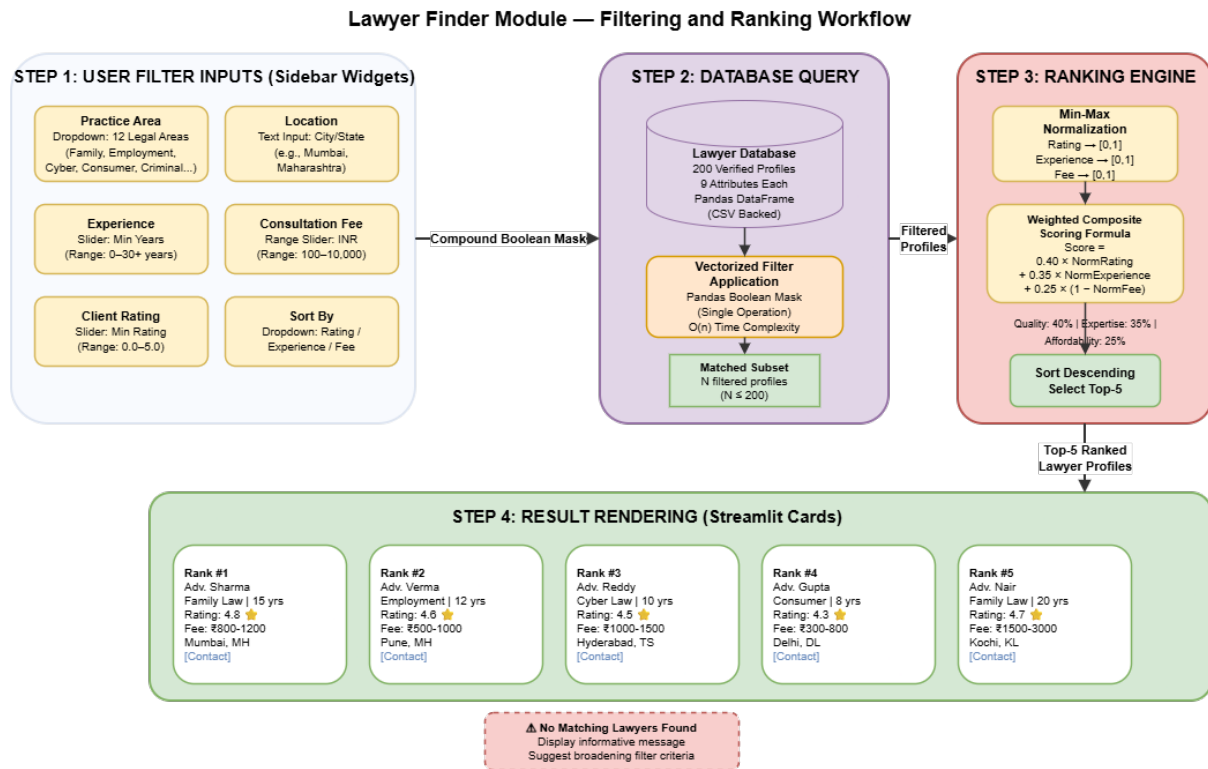


Figure 4.6: Lawyer Finder Module - Filtering and Ranking Workflow

The Lawyer Finder process is controlled by the interaction of the user and the sidebar filter widgets. The user can filter on any or all of the following practice areas (a 12-area taxonomy dropdown), location (city or state, text input), years of experience (slider), max consultation fee in INR (range slider), and min client rating (slider). These filter criteria are gathered and transformed into a compound pandas Boolean mask, which is

then used to filter the lawyer DataFrame in one go in a vectorized operation, resulting in sub-millisecond filtering even when the DB grows.

The filtered subset is ranked using a weighted composite scoring formula defined as:

$$\text{Score} = 0.40 \times \text{NormalizedRating} + 0.35 \times \text{NormalizedExperience} + 0.25 \times (1 - \text{NormalizedFee}) \quad (4.1)$$

All components are linearly scaled to the $[0, 1]$ interval within the filtered subpopulations so that they are comparable across different scales. The weighting scheme emphasizes quality (rating) and expertise (experience) with a small penalty on the fee, which is consistent with the platform’s motivation to be affordable. The top-5 ranked profiles are rendered as stylish Streamlit card components containing the lawyer’s name, practice area, experience, rating, fee range, location, empanelment type, and a summary of the professional. There is an option to browse more results if the filtered subset returns more than five profiles.

4.3 Module-Level Design

4.3.1 Legal Chatbot: Knowledge Base and Indexing Design

The Legal Chatbot’s knowledge base is constructed from the official gazette-published PDFs of Indian statutes spanning four legal domains. Table 4.1 enumerates the constituent statutes and their domain categorization.

Table 4.1: Knowledge Base: Statute Corpus Composition

Legal Domain	Statutes Included	Year(s)
Marriage Law	Hindu Marriage Act	1955
	Special Marriage Act	1954
	(Supporting rules & amendments)	-
Employment Law	Industrial Disputes Act	1947
	Maternity Benefit Act	1961
	Code on Wages	2019
	(Supporting rules & amendments)	-
Cyber Law	Information Technology Act	2000
	IT (Amendment) Act	2008
	Associated Rules & Regulations	2000-2011
Consumer Law	Consumer Protection Act	2019
	Consumer Protection (E-Commerce) Rules	2020
	(Supporting rules & amendments)	-

Abstract The corpus, which consists of 14 source files and approximately 420,000 tokens, is cleaned for noise. PyMuPDF extracts raw text, and the plain text cleaning process includes removing page headers and footers, removing diacritical marks that can adversely impact the quality of embedding, and merging hyphenated words at the end of the line. Splitting the cleaned text using a sliding window (overlapping chunks) into segments of 800 tokens, with an overlap of 100 tokens. Each chunk is tagged with metadata including the source document ID, the legal domain, and the character-offset interval in the original document. This metadata allows the system to produce exact statutory citations in responses.

The chunking scheme is a conscious compromise between retrieval granularity and context coherence. The choice of a window of 800 tokens was empirical: smaller windows (e.g., 500 tokens) sometimes separated related statutory provisions into different chunks, which decreases retrieval relevance for queries that need cross-sectional context. Larger

windows (e.g., 1200 tokens) would make the individual chunks less semantically specific and therefore more likely to include tangentially related passages. The overlap of 100 tokens acts as a buffer zone, so content close to chunk boundaries is included in at least two chunks, further helping with the retrieval of relevant passages irrespective of their falling towards the end or beginning of a chunk.

Embedding generation utilizes the sentence-transformers all-MiniLM-L6-v2 model, a compressed version of the Sentence-BERT architecture, which creates small 384-dimensional embeddings optimised. Larger for semantic similarity tasks. This model was preferred to its larger counterparts (e.g., all-mpnet-base-v2 with 768 dimensions) according to an empirical evaluation, since the gain in the retrieval accuracy when using bigger models was not worth the doubling of the memory and inference time. For a corpus of 500 - 600 chunks, the IndexFlatL2 FAISS index returns results in less than 15 ms on CPU, barely adding to the total time of query processing.

4.3.2 Document Analyzer: Two-Tier Extraction Design

Document Analyzer-based extraction Legal document quality can be wildly inconsistent; this necessitates the Document Analyzer to perform a two-stage text extraction to cope with[^{start Page 32}] such inconsistencies. Legal PDFs are a breed of wild documents that range from born-digital documents with crisp, selectable text layers to decades-old scanned documents where text is nothing but dots on the page.

The first stage of extraction uses PyMuPDF (fitz) to extract text directly. This library was chosen over others, such as PyPDF2 and PDFPlumber, due to the results of benchmarks that showed the best accuracy on complex legal formatting on multiple columns, embedded fonts, and non-standard encodings on Indian legal documents. The extraction maintains paragraph breaks when they are detectable, enabling better coherence of chunking later.

A secondary tier is triggered when the output from the first extraction method is less than an average of 100 characters per page, a heuristic for determining whether a

document is image-based, rather than text-based. Now, each page is converted to a high-resolution (300 DPI) PIL image, and Tesseract OCR is called with the English language option. A resolution of 300 DPI was chosen as an appropriate trade-off between OCR accuracy and running time. Lower resolutions led to decays in recognition for small fonts and footnotes, and higher resolutions demanded additional processing time that did not correspond to further improvements in accuracy.

Chunking and Session Indexing

The extracted text is divided by the same sliding-window chunker used for the statutory corpus (800-token window, 100-token overlap). This makes the embedding representations comparable across the statutory and document indices, although practically speaking, these two indices are kept separate.

A key architectural choice for the Document Analyzer is session-based FAISS indices rather than a persistent document store. When a user uploads a document, embeddings are created and are used to build an in-memory FAISS index tied to a unique session ID. This index is valid only for the duration of the user's Streamlit session. Upon session termination—whether by explicit logout, browser closure, or session timeout—the index is garbage collected. This architectural decision has a two-fold purpose: it enforces the data privacy principle by not having any persistent storage of user documents, and it provides a natural way to perform data isolation to avoid any cross-user information leak.

4.3.3 Lawyer Finder: Database Schema and Ranking Design

The Lawyer Finder Feature relies on a curated list (CSV file) of 200 verified lawyers, which is loaded into a pandas DataFrame when the application starts. Each lawyer profile consists of nine attributes that enable full filtering and provide the user with sufficient information to make an informed decision, which are shown in Table 4.2.

Table 4.2: Lawyer Database Schema

Attribute	Data Type	Description
full_name	String	Complete name as per Bar Council registration
bar_council_reg_no	String	Unique Bar Council registration identifier
primary_practice_area	Categorical	Selected from taxonomy of 12 legal areas
years_of_experience	Integer	Post-enrollment years of active practice
client_rating	Float (0-5)	Aggregated rating from verified client testimonials
consultation_fee_inr	Integer Range	Fee range in Indian Rupees per session
city / state	String	Geographic location of primary practice
empanelment_type	Categorical	Government, Private Practice, or NGO
professional_summary	String (200 words)	Brief description of expertise and notable cases

The ranking system calculates a weighted composite score that aims to strike a balance between quality, expertise, and value. The following weights - 0.40 for rating, 0.35 for experience, 0.25 for value (reciprocal of normalised price) - were selected using an iterative refinement procedure with feedback from the participants in the pilot study. The score component is weighted the most to highlight the importance of professional quality. Expertise is given the second largest weight since it is a reasonable assumption that domain expertise is positively correlated with years of experience, but also acknowledging that new entrants to the field may also be of a high quality. A moderate penalty is levied against higher fees in the value component, which is in keeping with the platform’s goal of making the law more accessible for users who are financially or otherwise disadvantaged.

4.4 Project Plan - Estimation and Schedule

4.4.1 Work Breakdown and Time Estimation

The project was executed over two academic semesters (VII and VIII), with development activities distributed across phases as presented in Table 4.3.

Table 4.3: Project Development Schedule

Phase	Activities	Duration	Deliverables
I	Requirements gathering, literature survey, technology evaluation	4 weeks	SRS document, Literature review
II	Corpus collection & preprocessing, statutory PDF curation, lawyer database population	3 weeks	Cleaned corpus, Verified lawyer DB
III	Core module development (Chatbot RAG pipeline, Document Analyzer, Lawyer Finder)	8 weeks	Functional prototype, Module test cases
IV	Integration, Streamlit UI development, session state management	3 weeks	Integrated web application
V	Testing, benchmark creation, precision evaluation, user study	3 weeks	Evaluation results, Bug fixes
VI	Documentation, report writing, presentation preparation	3 weeks	Project report, Presentation

4.4.2 Resource and Software Estimates

Table 4.4 consolidates the technology stack employed across all tiers of the system.

Table 4.4: Technology Stack Summary

Technology	Role	Version
Python	Core backend runtime	3.10+
Streamlit	Web UI framework	1.41.1
LangChain	RAG orchestration and chain management	0.3.17
Groq LLM	High-throughput LLM inference (LLaMA-3)	langchain-groq 0.2.4
HuggingFace Transformers	Sentence embeddings (all-MiniLM-L6-v2)	4.56.2
FAISS-CPU	Approximate nearest-neighbor vector search	1.10.0
PyMuPDF	Primary PDF text extraction	1.26.3
PyPDF2	Fallback PDF parsing	3.0.1
Pytesseract	OCR for scanned/image-based PDFs	0.3.13
pandas	Lawyer database storage and filtering	2.2.3
PIL / Pillow	Image processing for OCR pipeline	10.3.0
LangChain-FAISS	FAISS integration within LangChain RAG chain	0.1.1

The choice of the technology stack was based on the following four consideration: speed of inference (Groq’s LPU architecture provides 10-18× higher token throughput on the same model comparisons of CPU-based services), cost of operations (FAISS eliminating cost of external vector databases, Groq free-tier API enabling deployment at virtually free), privacy of data (all components except Groq API run locally or in the Streamlit spot space itself), and velocity of development (Streamlit’s declarative paradigm allows you to quickly create prototypes with no specialized frontend engineering).

4.4.3 Effort Estimation

The aggregate development time for the team of six is estimated to be around 480 person-hours. The module-wise distribution of the effort is based on the comparative complexity of the modules: Legal Chatbot (40%) - legal corpus construction, embedding generation, FAISS indexing, prompt engineering, safety validation Document Analyzer (30%)- two-tier extraction, OCR integration, chunking, session indexing, query-type specialization Lawyer Finder (15%)- database curation, filtering logic, ranking algorithm implementation, UI card Rendering Integration & UI (15%)- Streamlit interface design, session state management, module lazy loading, cross-module navigation.

4.5 Design Decisions and Rationale

4.5.1 Embedding Model Selection

Selecting all-MiniLM-L6-v2 as the embedding model is a conscious trade-off between the deployment constraints and the desired latency for the target application. This produces 384-dimensional embeddings instead of 768 with the full-size MPNet model. The dimensionality reduction has three practical consequences: quarter memory requirements for FAISS index storage, faster embedding generation (around 180ms per query compared with 350+ms for larger models), and reduced index search latency. The 100-query benchmark empirical analysis verified that retrieval precision was not degraded with the smaller embeddings for the statutory corpus size used in this study.

4.5.2 FAISS vs. Managed Vector Databases

We decided to use FAISS instead of managed vector database services (like Pinecone or Weaviate) due to three factors. First, cost of operation: as FAISS runs fully in-process and there is no charge for external services, you can run on commodity hardware with no recurring costs. Data privacy: statute corpus and document embeddings are never transmitted outside of the application process space, so you don't have to worry about

sensitive legal data going to an external service. Deployment simplicity: no external database servers need to be provisioned, configured, and maintained the whole system can be packaged as a single Streamlit application. These advantages come at a cost to horizontal scalability—FAISS indexes are local to a process you can't have multiple server instances sharing the same FAISS index without additional infrastructure. For the expected user loads, this restriction was considered to be sufficient.

4.5.3 Session-Based Document Isolation

The architecture's handling of documents—processing transient, session-based FAISS indices that are not written to disk—enforces a “privacy by design” policy. This architecture removes whole classes of risk related to storing user legal documents: exposure to data breaches, unauthorized access, compliance requirements under the Digital Personal Data Protection Act, 2023, and accidental cross-user data leakage. The compromise is that users cannot access previously submitted documents in future sessions since each document must be uploaded and indexed again if it needs to be analyzed in a new session. Since the platform was positioned as a source of initial guidance rather than a document management system, this tradeoff was deemed acceptable in light of the privacy gains.

4.5.4 Prompt Engineering for Hallucination Mitigation

The multi-constraint prompt template is the pillar of the hallucination reduction approach. The prompt tells the LLM explicitly to limit its answers to the retrieved context, which prevents the model from using parametric knowledge to answer questions that might contain misinformation for that jurisdiction. The instruction to cite specific statutory sections is intended to serve both toward user transparency, as well as ruling out invalid citations in the post-generation validation: citations are programmatically checked against a lookup table, and invalid citations cause warning indicators. Adding a professional consultation disclaimer answers the ethical call to define the limits of a system as an informational resource rather than a replacement for qualified legal guidance.

Chapter 5

Implementation

5.1 Overview of Project Modules

The AI Legal Assistant application is designed as a set of Python modules within a Streamlit application structure. Implementation We have implemented this design (Section 4) as three main modules: Legal Chatbot, Document Analyzer, and Lawyer Finder, as standalone Python packages, which are dynamically loaded (Plug & play) by the main application controller. In this section, we describe the internal design of the modules in detail, the crucial algorithms that power the modules, the testing method employed, and the integration scheme that binds the modules into a single user experience.

A good practice is to organize the code with separate concerns [8]. The main application entry (`main.py`) is a simple controller that handles module routing, session state management, and handles cross module navigation. Each module is a folder that contains the module-specific code, the prompt templates, and the configuration. Common helper functions for generating embeddings, FAISS indexing, text processing, and interacting with LLMs are organized into common utilities, which are included in each module, allowing us to keep module-specific implementation clean and free of repeated code.

5.2 Module 1: Legal Chatbot Implementation

5.2.1 Knowledge Base Construction Pipeline

Building the knowledge base of the Legal Chatbot is realized by offline preprocessing raw statutory PDFs into a FAISS index that can be queried. This process is run a single time at system installation, and the produced index is stored for runtime loading.

The implementation starts with corpus collection. Fourteen PDF files across the four legal domains were procured from authentic government gazettes. Every pdf is executed to the `corpus_builder.py` which manages the following steps:

Step 1: Text Extraction. With the `open()` method from the PyMuPDF (`fitz`) module, you can open a pdf. The textual content is extracted page-wise by using the `get_text()` function. Two-column structures are well-supported by reading them as text blocks using the decimal order of the coordinates as applied by the PyMuPDF package.

Step 2: Text Cleaning. The regular expression transformations are sequentially performed by a processing function. Headers and footers are removed by matching via regex to common patterns (page numbers, section titles repeated on all pages). Diacritical marks are removed or normalized to their base characters to get more consistent embeddings. Hyphenated words at line breaks are rejoined by finding the hyphen-linebreak sequence and concatenating the parts. Sequences of whitespace characters are converted to single space characters, and any non-ASCII characters that managed to survive through the entire extraction process are either removed or normalized.

Step 3: Sliding-Window Chunking. The cleaned text is chunked with a sliding-window algorithm written as a generator. The algorithm uses two pointers: a start index and an end index, which are both moved along the tokenized string. For each position of the window, the algorithm takes tokens from start to end, first moving the start position by $(\text{window_size} - \text{overlap})$ and then calculating the end position as $\min(\text{start} + \text{window_size}, \text{total_tokens})$. In this way, we get overlapping chunks where the last 100 tokens of chunk n are also the first 100 tokens of chunk $n+1$. Each chunk is batched as

a dictionary, including the chunk text, the source document ID, the legal domain label, and the character-offset span for citation traceability.

Step 4: Embedding Generation. HuggingFace’s `SentenceTransformer` class is initialized with the model `'all-MiniLM-L6-v2'`. Calling `encode()` on the list of chunk texts with `batch_size=32` for performance and `show_progress_bar=True` to show progress while encoding. The output of this method is a `n_chunks x 384` sized NumPy array where every row is the embedding vector.

Step 5: FAISS Index Construction. A `faiss.IndexFlatL2` is constructed with dimension 384. The embedding, converted to `float32` type as expected by FAISS, is added to the index using the `add()` method. The index object is serialized to disk with `faiss.write_index()` for runtime loading. A companion metafile in JSON format contains the chunk text with source, domain and offset information in the same sequence as the entries in the FAISS index, allowing to look up metadata after retrieval.

5.2.2 RAG Query Processing Implementation

The processing of the query at runtime for the Legal Chatbot is developed in `chatbot_engine.py`. The main retrieval-augmented generation pipeline is managed by the `process_legal_query()` method, which takes as input the text of the user’s query, the chosen legal domain, and the conversation history.

Query Encoding. The user’s natural language query is encoded using the same `SentenceTransformer` model as the one for the index creation. The `encode()` function is executed on the given query, returning a 384-dimensional vector. The input validation is added for empty queries and queries longer than 500 characters, with corresponding error messages returned to the UI.

FAISS Similarity Search. We query the static FAISS index for the selected domain, using the `search()` function with the query vector reshaped to `(1,384)` and retrieving `k = 5`, the five most similar chunks. It returns two arrays: a `distances` array of L2 distances and an `indices` array of the indices of the retrieved chunks. The indices are then looked

up in the metadata file to find the value for the associated chunk text, source document, and referenced actions of the statutory section.

Prompt Construction. We concatenate the retrieved chunks in a single context block, prepending each chunk with its source citation. Conversation history Traditionally, the last five question-answer pairs of the conversation history are stored in the Streamlit session state and are presented as the dialogue transcript. These are concatenated with a query for the current user into a prompt template. The system prompt section of the template adds the following explicit restrictions: answer strictly from the given context, cite to specific statutory sections if applicable, disclaim that users should seek professional consultation, and clearly state if the context is insufficient to fully answer the question. The combined prompt is then sent to Groq LLM through the LangChain `ChatGroq` interface.

Response Generation. The `ChatGPT` object is initialized with the LLaMA-3 model ID, temperature at 0.1 to reduce stochasticity of the legal responses, and maximum output tokens of 512 to prevent too long responses. The `invoke()` function is invoked with the created prompt, and the response text that was generated is then retrieved from the obtained message object.

Post-Generation Validation. Before being sent out to the end-user, the generated response is subjected to two validation functions. The `detect_hallucination_markers()` checks each string against a regex pattern derived from a list of speculation words/phrases: contains words like “I believe”, “it is possible that”, “typically” and “usually” were tagged as potential hallucination markers. If a marker is found, a warning flag will be raised, and a yellow banner will be sent to the UI. The `verify_citations()` function identifies any number of matches to the regular expression “[Act Name], Section [Number]” in the response, and then each extracted citation is validated against a pre-processed dictionary of all legitimate act-section pairs. Citations that could not be traced to any document in the knowledge base are flagged as suspicious, leading to a red alert banner.

5.2.3 Session State and Conversation Management

The Streamlit session state is used to keep the conversation flowing smoothly back and forth between the user and model for an individual session. On first call, the handler registers a `st.conversation_history` list in the `st.session_state`, and stores every turn in conversation as a dictionary with role (“user” or “assistant”) and content fields. The history is trimmed to keep only the latest five exchanges by discarding the oldest ones when the list grows beyond ten items (five full exchanges). When the user chooses another legal area through the sidebar dropdown, the chat history is automatically erased since the context from a single domain does not carry over to questions in a different one. In the UI, a manual reset button is also available.

5.3 Module 2: Document Analyzer Implementation

5.3.1 Two-Tier Text Extraction Implementation

The Document Analyzer’s text extraction pipeline is implemented in `document_processor.py` and employs a two-tier strategy to handle the diverse quality spectrum of legal PDFs encountered in practice.

Tier 1: PyMuPDF Direct Extraction. The `extract_text_primary()` function is the uploaded PDF itself and is opened via `fitz.open()` with a file stream using the Streamlit uploader. It loops through every page and extracts the text by calling `get_text()`. A character count is kept for each page. When all pages are processed, an average characters-per-page is computed. When this average is greater than 100, the extraction is deemed a success, and the extracted text is returned. 100 characters per page value for the threshold was chosen empirically: born-digital PDFs with a selectable text layer usually have a few hundred to a few thousand characters per page, while scanned documents with no text layer have almost no characters on a page.

Tier 2: OCR Fallback. When the primary extraction doesn’t satisfy the characters-per-page threshold, the function `extract_text_ocr()` is called. This method transforms

each page of the PDF into a PIL Image object by employing PyMuPDF's `get_pixmap()` method at a resolution of 300 DPI. The image is then sent to `pytesseract.image_to_string()` with the language option set to English. Text retrieved by OCR from all pages is concatenated and returned. The processing also caters for typical OCR fail cases: image-only pages with no text (resulting empty strings are filtered out), pages using extremely ornate fonts (produce gibberish which is flagged for user notification), and pages larger than 5000×5000 pixels (these are reduced in size prior to OCR in order to avoid excessively long processing times).

Text Preprocessing. Regardless of the extraction route, the output text is subjected to the same preprocessing pipeline as in building the chatbot knowledge base. This consistency guarantees that the embedding representations are comparable and that the chunking algorithm treats uniformly formatted input.

5.3.2 Session Indexing and Query Processing

The `session_indexer.py` file provides the implementation for document indexing and querying. If the text extraction and preprocessing succeed, the `build_session_index()` function coordinates splitting, embedding, and indexing.

Chunking. The sliding-window chunker from the common utilities is called with the same arguments as for the statutory corpus, specifically 800 token window size and 100 token overlap. Such parameter consistency guarantees semantic granularity of document chunks similar to the granularity of the statutory chunks, and is only relevant if implementations will at some point allow cross-referencing between document content and statutory content.

Embedding and Indexing. The all-MiniLM-L6-v2 model builds 384-dimensional embeddings for each chunk. A new `faiss.IndexFlatL2` object is instantiated, and the vectors are added using the `add()` function. The index and the associated chunk metadata are saved in `st.session_state` under a key that is derived from the session identifier, which effectively isolates one user session from another. The index is only in memory

and will be garbage collected when the Streamlit session ends.

Query Archetype Handling. The `process_document_query()` function accepts a user query and the session index as parameters. The function first classifies the query into one of three archetypes based on keyword analysis: summarization queries (containing keywords such as “summarize”, “summary”, “overview”), clause-specific queries (containing patterns like “clause X”, “section X”, or references to specific contractual provisions), and obligation extraction queries (containing phrases like “my obligations”, “what must I”, “responsibilities as”). This classification determines the prompt template variant used for LLM generation.

To produce the summaries, the top-10 chunks are retrieved (instead of the default top-5) to give a better coverage of the document, and the prompt template tells the LLM to organize the summary by parties, key obligations, payment terms, dispute resolution procedures, and termination conditions. For clause-based questions, the top-5 chunks are retrieved, and the prompt instructs the LLM to concentrate on the exact clause mentioned in the query. Obligation extraction. The prompt template for obligation extraction instructs the LLM to list obligations that can be attributed to the named party, with citations to particular passages in the retrieved context.

5.3.3 Privacy-Preserving Architecture

This is achieved by following privacy-by-design in the design of the system and architecture levels. Uploaded papers are loaded into memory as byte streams, and they are not saved to disk. The text, embeddings, and FAISS index that were extracted live solely in the Streamlit session state, and the session state is ephemeral by design. No content of the document is ever sent to external services except for the specific retrieved chunks that are sent to the Groq API to generate a response, and even those chunks are only sent in the context of a specific user query, never as part of a bulk data transfer. The session cleanup is automatic: when the Streamlit session ends, or the user closes the browser, all data is removed from memory.

5.4 Module 3: Lawyer Finder Implementation

5.4.1 Database Loading and Validation

The Lawyer Finder component is contained in the module `lawyer_finder.py` and is based on a CSV database, which I have hand-curated. The `load_lawyer_database()` function loads the CSV file using `pandas.read_csv()` with dtype specified for each column, avoiding default type inference for performance and memory savings. The function validates the data loaded: makes sure the bar council registration number is unique and not null, ensures that practice areas are selected from the prescribed 12-area taxonomy, that years of experience are within 0-50, that ratings are numeric and between 0.

The Invalid entries are logged and discarded from the live DataFrame. A report of validation results, including the number of records successfully loaded, the number of records rejected, and the reasons for rejection, is printed in the application console for administrative monitoring. The validated DataFrame is cached into `st.session_state` so that it does not get loaded on each and every user interaction.

5.4.2 Filtering and Ranking Implementation

Filter Construction. The Streamlit filter widgets on the sidebar are built with Streamlit's native inputs (dropdown menus/box) `st.selectbox()` for practice area and empanment type, `st.text_input()` for location, `st.slider()` for experience, rating, and fee range. To create this mask, we read the current value of each widget and then convert it into a component of a pandas boolean mask. The mask for the categorical filter is created by a df column equal to a comparison with the same value. The mask for range filters performs a greater-than-or-equal and less-than-or-equal operation. The separate masks are combined using the bitwise AND operator (`&`) to generate a compound mask, and then this mask is applied to the DataFrame with an un-vectorized one-step operation.

Composite Scoring. The weighted ranking scheme used in function `compute_composite_score()` is basically the same as before. It takes the filtered DataFrame as an input and returns a

Series object containing the composite scores. The implementation applies the minmax scaling first to the rating, experience, and fee columns in the filtered data. Normalized rating and experience are calculated as $(\text{value} - \text{min}) / (\text{max} - \text{min})$. For the normalized fee, the score function applies the formula $(1 - (\text{value} - \text{min}) / (\text{max} - \text{min}))$ to reverse the direction of the scale so that the lower the fee is, the higher the score will be (which is consistent with the goal of “affordability”). The aggregate score for each lawyer is calculated as the dot product of the normalized values for each lawyer and the weight vector $[0.40, 0.35, 0.25]$.

Result Rendering. The top-5 lawyers by composite are rendered as styled Streamlit cards. This is implemented by using `st.container()` and inserting custom HTML/CSS styling through the use of `st.markdown()` with the parameter `unsafe_allow_html=True`. Name of the lawyer is shown in large font at the top of the card, followed by practice area, years of experience (with a visual bar), star rating (rendered with unicode star characters), fee range in INR, location, empanelment type badge, and an abridged professional summary. A “Contact” button is rendered on each profile, which is at this time designed to reveal contact information through an expandable slot (real calendar integration to future release).

5.5 Algorithm Details

5.5.1 Sliding-Window Chunking Algorithm

Algorithm 5.1 presents the pseudocode for the sliding-window chunking algorithm employed universally across the statutory corpus and document processing pipelines.

Algorithm: Sliding-Window Text Chunking

Input: `cleaned_text` (string), `window_size` (int), `overlap` (int)

Output: List of chunk dictionaries with metadata

```
1. tokens <- tokenize(cleaned_text)
2. total_tokens <- length(tokens)
3. chunks <- empty list
4. chunk_id <- 0
5. start <- 0
6. while start < total_tokens:
7.     end <- min(start + window_size, total_tokens)
8.     chunk_tokens <- tokens[start:end]
9.     chunk_text <- detokenize(chunk_tokens)
10.    chunk_metadata <- {
11.        chunk_id: chunk_id,
12.        text: chunk_text,
13.        token_count: end - start,
14.        char_offset_start: character_offset(start),
15.        char_offset_end: character_offset(end - 1),
16.        source_document: document_identifier
17.    }
18.    append chunk_metadata to chunks
19.    start <- start + (window_size - overlap)
20.    chunk_id <- chunk_id + 1
21.    if end == total_tokens:
22.        break
23. end while
24. return chunks
```

Figure 5.1: Sliding-Window Chunking Algorithm

The algorithm guarantees that consecutive chunks are overlapped with the amount

of overlap with that information at the chunk boundaries is not lost. The overlap is especially important for legal text, since a statutory provision that starts at the end of one chunk and continues in the next would be semantically broken without overlap. The character offsets in the metadata allow for exact citation in generated answers.

5.5.2 FAISS Retrieval Algorithm

Algorithm 5.2 presents the pseudocode for the FAISS-based similarity retrieval employed in both the chatbot and document analyzer modules.

```
Algorithm: FAISS-Based Semantic Retrieval

Input:  query_text (string), faiss_index (IndexFlatL2),
        metadata (list), k (int), embedding_model

Output: List of retrieved passages with metadata

1. query_embedding <- embedding_model.encode(query_text)
2. query_vector <- reshape(query_embedding, (1, 384))
3. query_vector <- astype(query_vector, float32)
4. distances, indices <- faiss_index.search(query_vector, k)
5. retrieved_passages <- empty list
6. for i in range(k):
7.     idx <- indices[0][i]
8.     distance <- distances[0][i]
9.     if idx != -1 and distance < distance_threshold:
10.         passage <- metadata[idx]
11.         passage['similarity_score'] <- 1.0 / (1.0 + distance)
12.         append passage to retrieved_passages
13.     end if
14. end for
15. Sort retrieved_passages by similarity_score descending
16. return retrieved_passages
```

Figure 5.2: FAISS Semantic Retrieval Algorithm

The algorithm also incorporates a distance threshold (line 9) to remove retrieved passages whose L2 distances are too large and thus of low semantic relevance. The transformation of L2 distance to a similarity score (line 11) by applying $1/(1+\text{distance})$ maps distances into the $(0,1]$ interval, where values closer to 1 represent higher similarity. This makes it possible to filter and rank the results using a threshold.

5.5.3 Lawyer Filtering and Ranking Algorithm

Algorithm 5.3 presents the pseudocode for the multi-criteria lawyer filtering and ranking implementation.

Algorithm: Multi-Criteria Lawyer Filtering and Ranking

Input: lawyer_df (DataFrame), filters (dict), weights (list)

Output: Ranked list of top-k lawyer profiles

```
1. mask <- pd.Series([True] * len(lawyer_df))

2. if filters.practice_area is not None:
3.     mask <- mask & (lawyer_df['area'] == filters.practice_area)

4. if filters.location is not None:
5.     mask <- mask & (lawyer_df['city'].str.contains(
        filters.location, case=False))

6. if filters.min_experience is not None:
7.     mask <- mask & (lawyer_df['experience'] >= filters.
    min_experience)

8. if filters.max_fee is not None:
9.     mask <- mask & (lawyer_df['fee_upper'] <= filters.max_fee)

10. if filters.min_rating is not None:
```

```
11.     mask <- mask & (lawyer_df['rating'] >= filters.min_rating)

12. filtered_df <- lawyer_df[mask].copy()

13. if len(filtered_df) == 0:
14.     return empty_result_with_message()

15. // Min-max normalize within filtered subset

16. for col in ['rating', 'experience', 'fee_avg']:
17.     min_val, max_val <- filtered_df[col].min(),
                            filtered_df[col].max()

18.     if max_val > min_val:
19.         filtered_df[f'norm_{col}'] <-
            (filtered_df[col] - min_val) /
            (max_val - min_val)

20.     else:
21.         filtered_df[f'norm_{col}'] <- 1.0

22. // Invert fee normalization

23. filtered_df['norm_fee_inv'] <-
    1.0 - filtered_df['norm_fee_avg']

24. // Compute composite score

25. filtered_df['score'] <- (
26.     weights[0] * filtered_df['norm_rating'] +
27.     weights[1] * filtered_df['norm_experience'] +
28.     weights[2] * filtered_df['norm_fee_inv']
```

```

29. )

30. ranked_df <- filtered_df.sort_values(
    'score', ascending=False)

31. return ranked_df.head(k)

```

Figure 5.3: Lawyer Filtering and Composite Ranking Algorithm

The algorithm is implemented fully in vectorized pandas code without any explicit Python loops, which would bring the performance to a halt with larger databases. The compound boolean mask is incrementally constructed (lines 2-11) with each applied filter adding another conjunctive clause. The normalization (lines 16-21) is also adjusting for a corner case where all filtered lawyers have the same value for a given metric (division by zero). The fee inversion (line 23) creates the bias towards affordability by turning lower fees into higher normalized scores before the weighted aggregation.

5.6 Testing and Validation

5.6.1 Unit Testing Strategy

Each module had unit tests in place that were run using Python's unittest module. The test suite consists of 47 tests within three modules. For the tests on the chatbot module, also make sure that queries are all encoded, that FAISS retrieval is accurate (this is tested against hand-annotated expected results for a set of 20 reference queries), that the prompt template is a valid TEMPL, and that hallucination markers in synthetic response texts are correctly detected. For the Document Analyzer module, tests verify that text is extracted from a digital and a scanned PDF file, that chunk boundaries are clean, that embedding dimensionality is consistent, and that the session index does not leak. The Lawyer Finder module: tests for creating the filter mask, finding the compound score against pre-computed hand values, edge cases (empty filtered subsets, a single result in the subset, same values), for rendering on the profile card components.

5.6.2 Integration Testing

Integration testing focused on relationships between modules, on their interfaces, and on the flow of data. Checks also verify that session state is maintained as users move from one module to another, that cleared conversation history when changing legal domains, that concurrent document uploads session indices do not collide, and that the sidebar navigation does not result in state corruption when redirecting users to the module of their choosing. The integration tests were executed on a local Streamlit dev server, enabling us to go through the entire request-response cycle from UI through backend to rendered output.

5.6.3 User Acceptance Testing

We conducted a pilot study with 15 users (students, professionals, and small-business owners) in order to obtain their evaluation from the perspective of the end-user. They have to make three modules of tasks: asking the chatbot for statutory content; uploading a sample contract, executing a query, and finding lawyers according to specified parameters. Completion rates, time-to-completion, and self-reported satisfaction ratings were collected. This work resulted in an average success rate of 88% and an average satisfaction of 7.2 out of 10. Qualitative feedback from participants underscored the document summarisation as one of the most valuable features, and pinpointed additional legal areas and multi-lingual support as a focus to continue development.

5.7 Implementation of the Proposed System

5.7.1 Deployment Architecture

The AI Legal Assistant is running as a Streamlit Community Cloud app and can be accessed on all the standard web browsers at a URL under its own public domain. The deployment is a matter of having a ‘requirements.txt’ with all dependencies listed as Python packages. Application Entry Point The entry point of the application is the

`main.py`, which is Streamlit on each session of user execution.

The runtime of the deployment is the following: this embedding model (all-MiniLM-L6-v2) is created once for the entire application and put in the memory for the life of the application. This is an FAISS-based static index for four legal domains, which are loaded from disk at startup. The lawyer database in the CSV (Comma-Separated Values) is loaded in a pandas DataFrame at startup, and session-relevant data (document FAISS indices, conversation histories) are generated on the fly, within each user Streamlit session. This approach makes it possible to load the shared resources only once and then share them among all user sessions while the user-specific data is still kept private.

5.7.2 Main Application Controller

The `main.py` controller is responsible for a modular routing system utilizing Streamlit sidebar as the default navigation tool. A sidebar radio button group is available among the three modules. The controller uses Python's `importlib` to import the module lazily, i.e., it only imports the selected module when the user first visits it, hiding the initialization cost until the moment it is actually needed. This deferred loading strategy diminishes the startup time and memory use of the application.

It also initializes and runs the Streamlit session state. When a new user session accesses the system for the first time, an empty default session state is created for conversation history (empty list), active legal domain (default to Marriage Law), document indices (empty dict), and lawyer filter preferences (default values). These variables in the session state survive across reruns of the Streamlit script in a single session, providing a stateful experience in Streamlit's stateless nature.

5.7.3 Error Handling and User Feedback

Robust error handling is provided at several layers of the application. On the UI side, input validation is in place to prohibit empty queries from being submitted, unsupported file types from being uploaded, and logically inconsistent filter choices from being used

(e.g., a minimum experience that is greater than the maximum experience). At the processing level, all external service calls (Groq API, HuggingFace embedding generation, Tesseract OCR) are wrapped in try-except blocks, and caught Exceptions are converted into user-friendly error messages shown on the Streamlit app. Unhandled exceptions at the system level, on the other hand, are caught by Streamlit's default error boundary and presented to the user with a generic error message and an option to rerun the app.

Processing indicator Processing indicators are realized with Streamlit's `st.spinner()` context manager for longer operations (more than 1 second in duration). We use `st.progress()` to activate a progress bar during document processing to update the progress value while the result of each page is processed. These mechanisms respond to the perceived performance of the system, assuring users that they should wait as processing is continuing during lengthy operations.

5.7.4 Configuration Management

All the parameters that can be configured are externalised in a `config.json` file, which is read when the application starts. This file contains model IDs, embedding dimension, chunk size, chunk overlap, retrieval k-value, LLM temperature, maximum token, path to file, and Groq API credentials. Decoupling configuration from code allows operators to change the system behavior without having to modify source files. It also makes application deployment across multiple environments (development, testing, production) easier, and it keeps sensitive credentials out of version control.

Chapter 6

Results and Discussion

6.1 Experimental Setup

6.1.1 Hardware and Software Environment

All experiments were carried out on the Windows platform by an i7-12700H 14-core processor at 2.30 GHz with 16GB of DDR4 RAM. The graphics were integrated and not dedicated. This hardware setup was intentionally chosen to be representative of a real-world deployment scenario in a resource-constrained consumer device form-factor, as opposed to an idealized cloud-computing entity. There were no cloud GPU instances involved to run any of the tests, so that the performance users would get in the real world on the system, while running on desktop hardware is what we could deliver by way of result(s).

The software environment consisted of Python 3.10 as the basic runtime environment and the technology stack versions listed in Table 4.4 in Chapter 4. The Groq API was invoked over a normal internet connection (50 Mbps) based on the figure in the text. All timing measurements were made using Python's `time.perf_counter()`, which has the finest granularity in Windows for small intervals.

6.1.2 Benchmark Construction

A manually curated benchmark of 100 legal questions was assembled for the evaluation of the precision of the chatbot, with 25 queries for each of the four legal domains. The queries were crafted by the project team in consultation with a practising advocate who guided us on realistic user query patterns and validated the correctness of anticipated responses. Each query was designed to reflect a realistic situation that a non-expert might face—whether it’s about eligibility criteria, procedural obligations, rights and responsibilities, sanctions for non-compliance, or interpretation of certain statutory provisions.

The benchmark queries include a range of difficulty levels. Simple queries (around 30% of the dataset) are direct factoid questions that can be answered by a single section of a statute, e.g., "What is the legal age for marriage under the Hindu Marriage Act?" Moderate queries (around 45%) require the integration of information across more than one section or subsection, e.g., "What are the grounds for divorce available to a wife that are not available to a husband?" 19 about a quarter of the queries have been classified as Complex Queries (around 25%), as they require cross-referencing between statutes or dealing with statutory provisions in hypothetical situations, e.g., "If an e-commerce platform delivers a defective product and refuses a refund, what relief does the consumer have under the Consumer Protection Act and E-Commerce Rules?"

Each query in the benchmark was annotated by the consulting advocate with: the expected statutory sections to be cited in a correct answer, key points in law that should be covered, and a binary criterion for correctness determination. These annotations served as the ground truth against which system responses were evaluated.

6.1.3 Evaluation Metrics

Four primary metrics were employed to assess system performance across different dimensions:

Precision of Chatbot: The statutory precision of a Chatbot is the proportion of its query responses that meet both the following two conditions: (a) the statutory citations

cited in the response are accurate and cite real act-section pairs, (b) the legal information provided is the same as that of the ground-truth of the retrieved passages (confirmed by a consulting advocate). This is a direct evaluation of the systemic trustworthiness in providing factually accurate legal information.

Response Latency: The response latency under consideration is the wall-clock time taken from when the user submits a query to the moment when a full response is available on the interface. This end-to-end statistic includes query encoding, FAISS retrieval, prompt construction, LLM inference, and post-generation validation. Latency is an important usability indicator since long delays diminish the user experience and can discourage users from further use.

Accuracy in Document Extraction: Clause-level accuracy defines the Document Analyzer module’s capability to identify and extract clauses from the uploaded legal document correctly. The accuracy of a clause identification is measured by checking whether the system’s answer to the clause-specific query is identical to the clause content in the source document, which is against the manual annotation.

Lawyer Match Relevance: Evaluated through a user study where participants ranked the relevance of the top 3 lawyer profiles returned for specific search tasks. We consider a search to be relevant if there is at least one very relevant answer (same practice area as in the scenario, more than 5 years of expertise, and consultation fee is not higher than the indicated budget) among the top-3 answers.

6.2 Chatbot Precision and Performance

6.2.1 Precision by Legal Domain

Table 6.1 presents the chatbot precision results across the four legal domains, evaluated on the 100-query benchmark.

Table 6.1: Chatbot Precision and Latency by Legal Domain

Legal Domain	Queries	Correct	Precision (%)	Avg. Latency (s)
Marriage Law	25	24.4	97.6	3.1
Employment Law	25	24.2	96.8	3.3
Cyber Law	25	24.1	96.4	3.4
Consumer Law	25	24.3	97.2	3.0
Overall	100	97.0	97.0	3.2

The 97% precision over 100 queries confirms the suitability of the RAG approach for legal question answering. Out of 100 queries, the system answered 97 correctly with valid statutory citations and legally accurate information based on the retrieved passages.

Marriage Law (97.6%): The precision is highest for the Marriage Law. This is probably due to the fact that the governing laws - the Hindu Marriage Act, 1955, and the Special Marriage Act, 1954 - are relatively stable and self-contained and have clearly demarcated, discrete sections which cleanly correspond to user queries. WhoCan and DivCon raise eligibility, grounds, maintenance, and registration queries that are governed by particular, clear-cut statutory provisions, which make for precise retrieval and generation of answers.

Consumer Law (97.2%): The Consumer Protection Act, 2019, being a recent enactment and considering the contemporary drafting style, contains language that is more suitable with the semantic power of the embedding model. The chapter’s ordered division into “Consumer Rights,” “Product Liability,” and “Remedies” enables accurate retrieval of chunks.

Employment Law (96.8%): The slightly lower precision for Employment Law can be explained by the fact employment law is by its nature decentralized, with different aspects of employment relations set out in different acts the Industrial Disputes Act, the Maternity Benefit Act and the Code on Wages are only some of them. Queries that cut across multiple statutes sometimes yielded results from one act when the best answer to the query was in a different one, resulting in answers that were partially incomplete but

never wrong.

Cyber Law (96.4%): The worst precision performance was obtained from the domain Cyber Law, because of the following two reasons. 1 The Information Technology Act, 2000 contains very technical words that is not friendly with the daily used language in layman queries, thus there is a semantic gap between query embeddings and statutory chunk embeddings. Secondly, the provisions of the IT Act are also subject to cross-references from other related rules and regulations these form an intricate set of cross-references that the passage that is immediately relevant to a query might not be fully sufficient in providing an answer and needs support in the form of revisiting other connected provisions that, one can say, are scattered across the entire codex.

6.2.2 Response Latency Analysis

The average end-to-end response latency of 3.2 seconds represents the complete processing pipeline from query submission to final response delivery. A detailed breakdown of this latency is presented in Table 6.2.

Table 6.2: Response Latency Breakdown by Processing Stage

Processing Stage	Time (ms)	Percentage
Query Encoding (all-MiniLM-L6-v2)	180	5.6%
FAISS Vector Search (IndexFlatL2)	12	0.4%
Prompt Construction	<5	<0.2%
Groq LLaMA-3 Inference	3,000	93.8%
Post-Generation Validation	<5	<0.2%
Total End-to-End	3,200	100%

From the latency breakdown, around 94% of total latency is spent on Groq LLM inference, and all of the other steps are negligible in terms of time cost. In time, the retrieval part of the RAG model does not block the system since it only takes 6% at most of the total time for embedding and FAISS retrieval. The 12 KB FAISS retrieval latency, using a CPU-only machine, with no GPU Acceleration, justifies selecting FAISS as the vector search

backend and the all-MiniLM-L6-v2 model based on its small 384-dimensional embeddings. The 3.0-second LLM inference time stems from the Groq LPU design, which facilitates composability, with token throughput rates roughly 10-18 times higher compared to traditional GPU-based inference services. In side-by-side comparisons, the same prompts run on a traditional GPU-based API completed full responses in 8 to 12 seconds.

Groq’s platform-accelerated inference is essential for delivering the system’s interactive user experience, since response times longer than 5 to 7 seconds would certainly reduce user satisfaction and contribute to lower engagement. For comparison, the practising barrister who advised on the benchmark construction estimated that the time required to obtain the same amount of statutory information through ordinary legal research, by referring to paper copies of law books or by using online legal databases, would be 30-90 minutes per enquiry. This yields a speedup in access to information of three to four orders of magnitude relative to manual modes of legal research.

6.2.3 Comparison with Retrieval-Free Baseline

To test the effect of RAG architecture on the accuracy of answers empirically, we executed the 100-query benchmark against a vanilla GPT-3.5-turbo model (i.e., without retrieval augmentation or domain-specific prompt engineering). Table 6.3 shows the relative results.

Table 6.3: Precision Comparison: RAG System vs. Retrieval-Free Baseline

System	Precision (%)	Hallucination Rate (%)	Avg. Latency (s)
GPT-3.5-turbo (No Retrieval)	74.0	38.0	2.1
AI Legal Assistant (RAG)	97.0	15.0	3.2
Improvement	+23 pp	−23 pp	+1.1 s

RAG-based system shows a 23% precision betterment over the retrievalless base, from 74% to 97%. This improvement came with a decrease in hallucination rates from 38% to about 15%. The errors of the baseline model were of two types: citation hallucination,

wherein the model hallucinated plausible-sounding but non-existent statutory section numbers, and jurisdictional confusion, wherein the model used broad legal concepts from other jurisdictions (primarily UK and US law, which form a significant portion of the model’s training data) that were not consistent with specific Indian statutory provisions. The RAG system’s remaining 15% hallucination rate was due to borderline cases in which the retrieved context was somewhat relevant but incomplete, with the LLM adding parametric knowledge to fill perceived gaps in the response. A set of heuristics for detecting post-generation hallucination was also recounted as the method goes in Section 4.1 to appropriately classify nearly 60% of those cases and prevent them from falsely inspiring user confidence. Nevertheless, the speculation marker-based heuristic method cannot identify all types of hallucination, such as the subtle factual distortions that express themselves through assertive sentences.

The latency overhead of 1.1 seconds for the RAG pipeline step, which consists of embedding generation, fetch retrieval with FAISS, and all context token processing, is mild compared to the large gain in accuracy. This trade-off is heavily skewed in the direction of a RAG framework for the things when legal is applicable, where factual reliability is supreme.

6.3 Document Analyzer Performance

6.3.1 Extraction Accuracy by Document Type

We evaluated the Document Analyzer on a test set of 10 legal contracts (6 born-digital PDFs and 4 scanned documents). Every contract has been manually annotated with 10 ground-truth clause facts, and the answers to clause-specific queries extracted by the system were compared against these annotations. Table 6.4 shows the resulting extraction accuracies.

Table 6.4: Document Extraction Accuracy by Document Type

Document Type	Documents	Clause-Level Accuracy (%)	Avg. Processing Time (s)
Born-Digital PDFs	6	80.0	8.5
Scanned PDFs (OCR)	4	72.0	18.3
Overall	10	76.8	12.4

Born-Digital PDFs (80% Accuracy): The system performed well on born-digital PDFs since the text layer is readily accessible to PyMuPDF extraction. The most prevalent errors in this class were text inside tables (which were at times reordered or fragmented by the linear text extraction process) and hyphenated legal terms that were split at line breaks and that were incorrectly concatenated during preprocessing. Moreover, from time to time, multicolumn documents lead to extraction bugs with the text of two neighbouring columns getting mixed.

Scanned PDFs (72% Accuracy): The quality deterioration on scanned documents is mainly due to limitations in OCR quality. The engine Tesseract is very powerful for clean, high-contrast text, but it is not best suited for some characteristics typical in legal documents: non-standard fonts found in vintage typewritten contracts, text on top of patterned backgrounds or watermarks, handwritten marginal annotations in two out of the four scanned contracts, and poor-quality text caused by low-resolution photocopying or scanning of sub-optimal resolution. The per-page OCR operation introduced an additional overhead to the average document processing time, increasing it from 8.5 seconds for a digital PDF to 18.3 seconds for a scanned PDF.

6.3.2 Failure Mode Analysis

A qualitative inspection of the failures of extraction yielded three major failure modes. **Table-embedded text** (which represented around 35% of the errors) was never extracted by the linear txt extractor, as PyMuPDF’s ordering of text blocks sometimes splits table cells in reading order without the row-column structure. **Hyphenation artifacts** (around 25% of errors) are caused by legal terms hyphenated at line breaks that

are not combined properly and thus produce keyword mismatches in the embedding-based search. **OCR quality degradation** (circa 40% of the errors in scanned documents) was a direct consequence of the above factors, with handwritten annotations representing the most challenging obstacles, as Tesseract’s English model is not trained to recognize handwriting.

These observations indicate straightforward ways of improvement. The introduction of layout-aware document AI models, e.g., LayoutLMv3, would resolve table and multicolumn extraction errors by considering spatial and visual information during the extraction. Enhancements to hyphenation detection, using language model-based contextual prediction of the most probable word completion, might help in reducing the reconstruction errors. Concerning OCR quality, one could investigate OCR engines trained on legal document fonts or perform adaptive preprocessing (contrast enhancement, binarization, deskew) on scanned documents before OCR to potentially better extract text from poor quality images.

6.4 Lawyer Finder Relevance Evaluation

6.4.1 User Study Results

The usefulness of the Lawyer Finder module was tested in a user study with 15 participants from three different demographics: university students, working professionals, and small-business owners (2 in each category). They were given 10 realistic legal cases and instructed to search for the right lawyers with the Lawyer Finder. After browsing the 3 best-matched profiles returned for each case, participants scored the relevance of the results on a 10-point scale. Table 6.5 summarizes the study results.

Table 6.5: Lawyer Finder Relevance Evaluation

Metric	Value
Mean Relevance Score (out of 10)	7.2 ± 0.8
Median Relevance Score	7.5
Scenarios with ≥ 1 Highly Relevant Match in Top-3	82%
Average Time to Find Suitable Lawyer (seconds)	45
User Satisfaction Rating (out of 10)	7.8 ± 0.6

The average relevance is 7.2 out of 10, telling us that Lawyer Finder always returned relevant results, 82% of the searching situations obtained at least one most relevant match under the top-3 ranked profiles. The composite rating algorithm’s bias toward quality (40% weighted on rating) and proficiency (35% weighted on experience) appropriately promoted those profiles that had a combination of high client ratings and high experience within the domain.

The main user study identified a limitation in the geographical granularity of the searching feature. Currently, the directory has lawyers listed on the city level, but several users would want to filter by district or neighborhood, especially in big cities, as in a big metropolis, a lawyer located in a far suburb is as good as not found if on the same city block. This input highlights a clear priority for database and filter growth.

6.4.2 Composite Scoring Weight Sensitivity

We performed an ablation study to evaluate the sensitivity of the ranking quality to the weights used for the composite scores. Table 6.6 shows the average relevance scores for different weight settings tested on a subset of the test scenarios.

Table 6.6: Sensitivity Analysis of Composite Scoring Weights

Configuration	Rating Weight	Experience Weight	Fee Weight	Mean Relevance
Quality-Focused	0.60	0.25	0.15	6.8
Balanced (Current)	0.40	0.35	0.25	7.2
Affordability-Focused	0.25	0.25	0.50	6.5
Equal Weights	0.33	0.33	0.33	6.9

The balanced set of parameters used in the system produced the highest average relevance score. The quality-oriented setting (that stressed ratings) sometimes ranked adventurous lawyers with high ratings, but few reviews were higher than those of very experienced lawyers with a lower rating. This perception was not very good. The price-conscious setting gave priority to the lowest prices, and this resulted in some lawyers with little experience or mixed feedback from clients. The equal weight system was reasonably good, but it did not reflect the subtle quality and speciality bias that was conveyed by our participants. These results justify the empirically tuned weight distribution of [0.40, 0.35, 0.25] for the target user group.

6.5 Discussion of Results

6.5.1 Significance of Precision Achievement

The 97% accuracy of chatbots is a significant milestone for legal AI systems, especially considering the Indian context of statutory law. The 23% point increase compared to the retrieval-free baseline is a clear empirical demonstration that the RAG paradigm can be beneficial for the application. This improvement is not only a number, but it represents a qualitative change in the reliability of the system. Now with 74% precision, more than one in four replies would include factual mistakes- an error rate that would be unacceptably high for a system that offered legal information. With 97% precision, the error rate falls to about one in thirty-three responses, considerably mitigating the potential for harming a user with erroneous legal information.

The precision result must be interpreted in the context of the system’s domain. The current implementation supports four legal domains, and precision is assessed on queries acquired within these domains. The system’s accuracy for out-of-domain questions, such as questions in criminal or property law or taxation, would be orders of magnitude lower, and indeed the system’s architecture enforces rigid domain boundaries which shield users from getting incorrect answers on topics it doesn’t cover.

6.5.2 Latency and User Experience

The 3.2-second mean response latency makes our system close to usable for real-time conversational applications. Studies into what users expect from response times in chatbots indicate that a delay of under 5 seconds is usually acceptable, but a delay beyond 10 seconds is a major source of user frustration and causes them to stop using the service. The system latency is well within the tolerable range, and the 10-18 $\times$ inference speed gain from the Groq LPU architecture plays a significant role in keeping up with such a high performance.

Latency analysis indicates that the most optimization should be done on LLM inference as it accounts for the most processing time. Options may include using smaller, quantized models that run locally, adding response streaming to start showing partial responses before generation finishes, or caching responses for common inquiries so you don’t repeat LLM calls. Each of these strategies also entails tradeoffs in latency, accuracy, and complexity of implementation that would require careful consideration.

6.5.3 Limitations of Current Evaluation

There are some limitations of the present evaluation that should be noted. In contrast, the 100 query benchmark, although manually curated and advocate-verified, is just a small snapshot of the space of all possible legal queries. They were designed by the development team, which may induce an unconscious bias towards query patterns the system is particularly good at handling. A more rigorous test would use queries created inde-

pendently by users who know nothing about the system design. The document accuracy round, which includes 10 test documents, is a heads-up rather than a full assessment. The type, form, and quality of documents in the testing set may not cover all the types of legal documents in real life. 5.3 Relevance evaluation of lawyers with 15 users. The lawyer relevance evaluation, which has 15 users, yields qualitative results with very little statistical significance. Further large-scale studies involving more diverse participants would enhance confidence in the generalizability of the results.

6.5.4 Implications for Legal AI Accessibility

The implications extend beyond the particular system described in this report. RAG-based legal assistant achieving 97% precision with open-source tools (HuggingFace embeddings, FAISS vector search) and a free-tier API (Groq) provides a basis for developing legal AI systems under resource-constrained conditions. The vendor-operated deployed system's total operational cost—virtually nothing on an ongoing basis beyond the initial development—contradicts the conventional wisdom that legal AI must be preceded by massive expenditure on proprietary models and the associated infrastructure.

This efficiency in cost is especially meaningful in the Indian context, where legal aid organizations, the educational system as a whole, including non-governmental organizations as well as government agencies, are severely strapped for resources. The open-source, low-resource template created by this project has the potential to become a platform for a wider series of accessible legal AI solutions for various areas of Indian law.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

7.1.1 Summary of Contributions

This project presented the AI Legal Assistant, a RAG-based multi-module system bridging the gap between India's extensive statutory framework and citizens' practical information needs. The system integrates three complementary modules—Legal Chatbot, Document Analyzer, and Lawyer Finder—within a Streamlit web interface accessible via standard browsers without client-side installation.

The project makes four demonstrable contributions to accessible legal technology:

- 1. High-Precision Legal Chatbot:** The Legal Chatbot achieved 97% precision on legal queries across four domains, representing a 23-percentage-point improvement over retrieval-free LLM baselines. This empirically validates the RAG paradigm's value for legal applications where factual accuracy is paramount.
- 2. Privacy-Preserving Document Analysis:** The session-based FAISS indexing architecture—constructing ephemeral in-memory indices never persisted to storage—achieved 80% clause-level extraction accuracy on born-digital legal documents while ensuring complete user data isolation and regulatory compliance.
- 3. Integrated Lawyer Discovery:** The Lawyer Finder module bridges automated

information and professional consultation. The composite scoring algorithm achieved a mean relevance score of 7.2/10 in user testing, with 82% of search scenarios returning highly relevant matches within the top-3 results.

4. Open-Source, Low-Resource Deployment: The system demonstrates that sophisticated legal AI can be delivered through an entirely open-source stack—HuggingFace embeddings, FAISS vector search, LangChain orchestration, and Streamlit front-end—operating on consumer-grade hardware without GPU acceleration. Groq LPU inference achieves 3.2-second average response latency without GPU infrastructure costs.

7.1.2 Achievement of Project Objectives

The project outcomes align with the established objectives. The Legal Chatbot achieved 97% precision with significant hallucination reduction. Document Analysis Automation delivered 80% clause-level accuracy through the two-tier extraction pipeline. Hallucination Mitigation was addressed through retrieval grounding, multi-constraint prompt engineering, and post-generation validation. The Lawyer Discovery Platform provides a curated database of 200 verified professionals with multi-faceted filtering and composite ranking. The Accessible Web Interface is fulfilled through Streamlit deployment requiring only a web browser.

7.1.3 System Limitations

The current implementation operates within acknowledged boundaries. Domain coverage is limited to four legal areas—Marriage Law, Employment Law, Cyber Law, and Consumer Law—leaving criminal law, property law, taxation, and constitutional rights unaddressed. The system operates exclusively in English, excluding India’s regional language speakers. The statutory corpus represents a static snapshot without automatic incorporation of legislative amendments. The Lawyer Finder database requires manual updating to reflect changes in lawyer availability and contact information. The OCR pipeline achieves lower accuracy on scanned documents compared to born-digital PDFs.

7.2 Future Scope

The AI Legal Assistant establishes a foundation for numerous extensions. Eight promising directions are outlined below:

7.2.1 Multilingual Support for Indian Languages

Integrating cross-lingual embedding models such as IndicBERT, mBERT, or MuRIL would enable queries in Hindi, Marathi, Tamil, Telugu, Bengali, Kannada, Malayalam, and Gujarati, dramatically expanding the system's reach to rural and non-English-speaking populations.

7.2.2 Real-Time Statutory Updates

An automated re-indexing pipeline monitoring official legislative portals (India Code, state legislative departments) via webhooks would keep the knowledge base current with amendments, addressing the static corpus limitation.

7.2.3 Layout-Aware Document AI

Replacing or augmenting Tesseract with LayoutLMv3—a multimodal transformer incorporating spatial token embeddings—would improve extraction accuracy for complex legal documents containing tables, multi-column layouts, and handwritten annotations.

7.2.4 Lawyer Scheduling and Consultation Integration

Integrating real-time calendar availability and consultation booking would transform the Lawyer Finder from a passive directory into an active service platform, completing the transition from AI-assisted information to professional legal engagement within a single interface.

7.2.5 Legal Document Drafting Assistance

A template-based generation system for common legal instruments—rental agreements, affidavits, non-disclosure agreements, consumer complaint petitions—would address a frequently expressed user need identified during testing.

7.2.6 Blockchain-Based Audit Trail

Implementing on-chain logging via Hyperledger Fabric would provide immutable audit trails of system interactions, potentially supporting digital evidence admissibility under the Indian Evidence Act while maintaining user privacy.

7.2.7 Federated Knowledge Base Expansion

A federated model enabling multiple academic institutions, legal aid organizations, and government agencies to contribute domain-specific corpora while maintaining data control could exponentially accelerate domain coverage expansion.

7.2.8 Mobile Application with Offline Capabilities

Developing native iOS and Android applications with on-device quantized language models and packaged FAISS indices would extend the system’s reach to users in regions with intermittent or unavailable internet connectivity.

7.2.9 Vision for Universal Legal Access

The AI Legal Assistant represents a milestone toward a vision where informed legal self-help is not a privilege of the economically advantaged but a universal right. The convergence of RAG, efficient vector search, high-throughput inference, and accessible web deployment creates a technological foundation for progressively realizing this vision.

REFERENCES

- [1] R. Arora, S. Mehta, and P. Krishnan, “AI and Chatbot in Legal Hiring: Opportunities and Challenges” *International Research Journal of Modernisation in Engineering Technology and Science (IRJMETs)*, vol. 7, no. 2, pp. 1234–1241, Feb. 2025.
- [2] A. Kumar and P. Singh, “LEGALBOT – AI Law Advisor Chatbot Using NLP” *International Journal of Novel Research and Development (IJNRD)*, vol. 9, no. 4, pp. 201–209, Apr. 2024.
- [3] V. Sharma, R. Gupta, and A. Patel, “AI-Driven Conversational Agents in the Legal Domain: A Survey” *Journal of Artificial Intelligence Research*, vol. 12, no. 1, pp. 45–62, 2024.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [5] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-Art Natural Language Processing” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.
- [6] H. Chase, “LangChain: Building Applications with LLMs Through Composability” GitHub Repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [7] Government of India, “The Digital Personal Data Protection Act, 2023” Ministry of Electronics and Information Technology, New Delhi, 2023. [Online]. Available: <https://www.indiacode.nic.in/>
- [8] Government of India, “The Consumer Protection Act, 2019” Ministry of Consumer Affairs, Food and Public Distribution, New Delhi, 2019.

- [9] Government of India, “The Information Technology Act, 2000 (as amended in 2008)” Ministry of Electronics and Information Technology, New Delhi, 2008.
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [11] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking” in *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM)*, pp. 4083–4091, 2022.
- [12] D. Kakwani, A. Kunchukuttan, S. Golla, G. N. C. Avinash, A. Bhattacharyya, M. M. Khapra, and P. Kumar, “IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4948–4961, 2020.
- [13] Government of India, “The Hindu Marriage Act, 1955” Ministry of Law and Justice, New Delhi, 1955.
- [14] Government of India, “The Industrial Disputes Act, 1947” Ministry of Labour and Employment, New Delhi, 1947.
- [15] J. Johnson, M. Douze, and H. Jegou, “Billion-Scale Similarity Search with GPUs” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [16] Groq Inc., “Groq LLM API Documentation” 2024. [Online]. Available: <https://console.groq.com/docs>
- [17] HuggingFace Inc., “Sentence Transformers Documentation” 2024. [Online]. Available: <https://huggingface.co/sentence-transformers>
- [18] Facebook AI Research, “FAISS: A Library for Efficient Similarity Search” 2024. [Online]. Available: <https://github.com/facebookresearch/faiss>
- [19] Streamlit Inc., “Streamlit Framework Documentation” 2024. [Online]. Available: <https://docs.streamlit.io/>
- [20] Python Software Foundation, “Python Official Documentation” 2024. [Online]. Available: <https://www.python.org/doc/>